



UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

**CEF 440: INTERNET PROGRAMMING AND MOBILE
PRORAMMING**

TASK7: FINAL PRESENTATION

GROUP 19

**Design and Implementation of a Mobile App for Collection of
User Experience Data from Mobile Network Subscribers**

GROUP MEMBER	MATRICULATION NUMBER
DJEUGWA WEAH MAXIMILIENNE	FE23A036
TCHOUANMOU TOUONANG WHITNEY ELSA	FE23A165
LOREL EKAMBA AKO-TABE	FE23A083
ESSOH RODDICK ESSOH	FE23A048
BEZIA PRECIOUS	FE23A026

COURSE INSTRUCTOR: Dr NKEMENI VALERY

ABSTRACT

This project presents the design and implementation of a cross-platform mobile application that measures and predicts the Quality of Experience (QoE) of Internet users. The application collects network performance indicators including download speed, upload speed, latency, jitter, signal strength, and network type. These measurements are analyzed and stored locally and remotely to provide users with historical reports and QoE predictions. The application was developed using React Native for the frontend, Node.js for the backend, and MongoDB for data storage. The completed application enables users to evaluate network quality, compare historical performance, and make informed decisions about their Internet connectivity. Testing results demonstrate that the application operates efficiently with accurate data collection and a user-friendly interface.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS	5
I. INTRODUCTION	6
1.2 Problem Statement	6
1.3 The Aim of The Project.....	7
1.4 Objectives of the Project	7
1.5 Scope of the Project.....	7
1.6 Significance of the Study of the Study	8
1.7Development Methodology	8
II. Requirement Gathering.....	9
1. Overview.....	9
2. Stakeholder Identification	9
3. Requirement Gathering Techniques.....	9
4. Data Gathering	16
5. Data Cleaning	18
6. Observations from Collected Data	19
7. User Reluctance Assessment.....	20
III. Requirement Analysis	22
2.1 Review and Analysis of Gathered Requirements	22
2.2 Identification of Inconsistencies and Missing Information	22
2.3 Requirement Prioritization	23
2.4 Requirement Classification	23
2.5 Validation of Requirements with Stakeholders	24
3. Software Requirements Specification (SRS).....	24
3.1 Purpose of the System	24
3.2 Scope of the System	24
3.3 Stakeholders and Users	25
3.4 Functional Requirements.....	25
3.5 Non-Functional Requirements	26
3.6 System Features.....	26
3.7 Hardware and Software Requirements	26
3.8 Constraints and Assumptions	27
3.9 Security Requirements	27
IV. System Modelling and Design	28
1. Context Diagram	28
2. Data Flow Diagram	31
3. USE CASE DIAGRAM	34
4. SEQUENCE DIAGRAM	36

5. Class Diagram	42
6. Deployment Diagram	44
V. UI DESIGN AND IMPLEMENTATION	51
Application Name: NetPulse	51
Tagline: "Measuring Network Experience, Empowering Connectivity."	51
Application Purpose	51
Target Users	51
Brand Identity	51
Logo Concept	51
5.2 Visual Design	52
5.3 Frontend Implementation	54
Demo Video	75
VI. DATABASE DESIGN AND IMPLEMENTATION	58
1 Objective	58
3 Conceptual Design	59
Relationship Analysis	59
4 Entity Relationship Diagram (ERD)	60
5 Database Implementation	63
6 Backend Implementation	65
7 Connecting Database to Backend	66
8 Challenges Encountered	67
VII. TESTING RESULTS AND DEPLOYMENT	68
6.1 Testing	68
6.2 Results	70
6.3 Deployment	70
VIII. MARKETING STRATEGY AND BUSINESS MODEL	71
IX. CONCLUSION	75
REFERENCES	77

LIST OF ABBREVIATIONS

QoE – Quality of Experience

QoS – Quality of Service

API – Application Programming Interface

HTTP – HyperText Transfer Protocol

REST – Representational State Transfer

UI – User Interface

UX – User Experience

DBMS – Database Management System

ER – Entity Relationship

SRS – Software Requirement Specification

I. INTRODUCTION

1.1 Background of the Study

The rapid growth of mobile technology and Internet services has transformed the way people communicate, work, study, and access digital information. As smartphones become increasingly powerful and affordable, mobile applications have become essential tools for activities such as online learning, video conferencing, entertainment, e-commerce, and social networking. The quality of these services largely depends on the performance of the underlying network connection.

Traditionally, network performance has been evaluated using **Quality of Service (QoS)** metrics such as download speed, upload speed, latency, jitter, and packet loss. Although these technical measurements provide useful information about network performance, they do not always reflect the actual experience of users. This has led to the adoption of the concept of **Quality of Experience (QoE)**, which focuses on users' perception of the quality and reliability of Internet services.

Several mobile applications, including Speedtest by Ookla, nPerf, and OpenSignal, allow users to measure network performance. However, many of these applications primarily display technical metrics without providing comprehensive analysis of the user's overall Quality of Experience or personalized recommendations based on the collected data.

This project presents the design and implementation of a **QoE Network Analyzer**, a cross-platform mobile application developed to monitor network performance and evaluate users' Quality of Experience. The application measures key network parameters such as download speed, upload speed, latency, jitter, and network type, stores test results, and presents them through an intuitive user interface. It also enables users to monitor network performance over time and make informed decisions regarding their Internet connectivity.

The project follows the complete mobile application development process, including requirement gathering, requirement analysis, system design, user interface development, database implementation, application testing, and deployment. The resulting application demonstrates the practical application of mobile development principles while providing a useful tool for monitoring and analyzing Internet performance from the user's perspective.

1.2 Problem Statement

Reliable Internet connectivity has become essential for activities such as online learning, video conferencing, cloud computing, online gaming, social networking, and digital commerce. Despite improvements in network infrastructure, users frequently experience inconsistent Internet performance that negatively affects the quality of these services. Existing network testing applications primarily display raw technical measurements such as download speed, upload speed, and latency without providing sufficient interpretation of how these values influence the user's overall experience.

Additionally, many available applications offer limited historical analysis, lack intelligent QoE evaluation, and provide few recommendations to help users understand or improve their

network performance. As a result, users often struggle to determine whether their Internet connection is suitable for specific applications or to identify patterns in network degradation over time.

There is therefore a need for a mobile application capable of combining network performance measurements with Quality of Experience analysis, historical reporting, and user-friendly visualizations. Such a solution would enable users to make informed decisions regarding their Internet connectivity while providing valuable data for researchers and network service providers.

1.3 The Aim of The Project

The aim of this project is to design, develop, implement, and evaluate a cross-platform mobile application that measures network performance and assesses the Quality of Experience (QoE) of Internet users through the collection, storage, analysis, and visualization of network performance metrics.

1.4 Objectives of the Project

The specific objectives of this project are to:

- Design an intuitive and user-friendly mobile application interface for network performance monitoring.
- Measure important network performance parameters including download speed, upload speed, latency, jitter, and network type.
- Store network test results locally and remotely for future reference.
- Analyze collected network data to estimate the Quality of Experience of users.
- Present historical network performance using graphical reports and statistics.
- Provide users with meaningful insights and recommendations based on measured network conditions.
- Implement a secure backend system for data storage and synchronization.
- Test, evaluate, and deploy the mobile application using industry-standard development practices.

1.5 Scope of the Project

This project focuses on the development of a cross-platform mobile application capable of measuring Internet network performance and evaluating the Quality of Experience of users. The application allows users to register, authenticate, perform network tests, store and retrieve historical test results, visualize performance trends, and receive QoE assessments.

The project covers the complete software development lifecycle, including requirement engineering, system modelling, user interface design, frontend implementation, backend development, database integration, testing, and deployment. The application is intended for Android devices and can be adapted for iOS with minimal modifications due to its cross-platform architecture.

The project does not include Internet Service Provider infrastructure management, network hardware configuration, or direct optimization of communication networks. Instead, it focuses on monitoring, analyzing, and presenting network performance information from the perspective of end users.

1.6 Significance of the Study

This project is beneficial to several categories of stakeholders. Individual users can better understand the quality of their Internet connections and determine whether their networks meet the requirements of activities such as streaming, online gaming, remote work, and virtual learning. Researchers can use the application as a platform for collecting and analyzing network performance data in different environments. Internet Service Providers may gain valuable insights into customer experiences, helping them identify areas requiring infrastructure improvements. Furthermore, students and software developers can use the project as a practical example of integrating mobile application development, networking, database systems, and software engineering principles into a real-world solution.

1.7 Development Methodology

The development of the QoE Network Analyzer followed a project-based software development methodology aligned with the Mobile Application Development course progression. The project began with requirement gathering through stakeholder identification and analysis of user needs. The collected requirements were reviewed and documented in a Software Requirement Specification (SRS). System modelling techniques, including UML diagrams, were then used to design the application's architecture and interactions.

Following the design phase, the user interface was developed using a modern cross-platform mobile framework, while the backend services and database were implemented to support data storage, user authentication, and synchronization. The completed application was subjected to unit testing, integration testing, system testing, and user acceptance testing to verify functionality, usability, reliability, and performance. Finally, the application was prepared for deployment and evaluated based on its ability to satisfy the identified project objectives and user requirements.

II. Requirement Gathering

1. Overview

In this requirement gathering phase, we aimed to bridge this gap by collecting both **subjective user feedback** and **objective network performance data**. We combined survey-based insights with measurements obtained from tools such as the Speedtest application. This report presents the stakeholders involved, techniques used, data collection and cleaning processes, user behavior analysis, and key observations that will guide the design and implementation of the proposed mobile application.

2. Stakeholder Identification

Identifying stakeholders was a crucial first step in our requirement gathering process, it ensured that all relevant perspectives were considered. The stakeholders for this system include:

Primary Stakeholders

- **Mobile Subscribers (Users):** Provide feedback and allow passive data collection
- **Mobile Network Operators (MTN, Orange, Camtel):** Use collected data to improve network performance
- **System Developers (Us):** Design and implement the application

Secondary Stakeholders

- **Researchers/Data Analysts:** Use data for QoE analysis and insights

We also considered the importance of **meeting with network providers**, as they are key beneficiaries of the system. These meetings would help align our data collection with industry needs.

3. Requirement Gathering Techniques

We used multiple techniques to ensure comprehensive data collection:

a. Surveys (Primary Method Used)

https://docs.google.com/forms/d/e/1FAIpQLSdyTjhxLZghjgc0bwADUty7n-SeULA_7nCkLEAgnk-r2LNwmQ/viewform?usp=preview

We designed and distributed questionnaires to mobile users to understand and to collect user perceptions regarding:

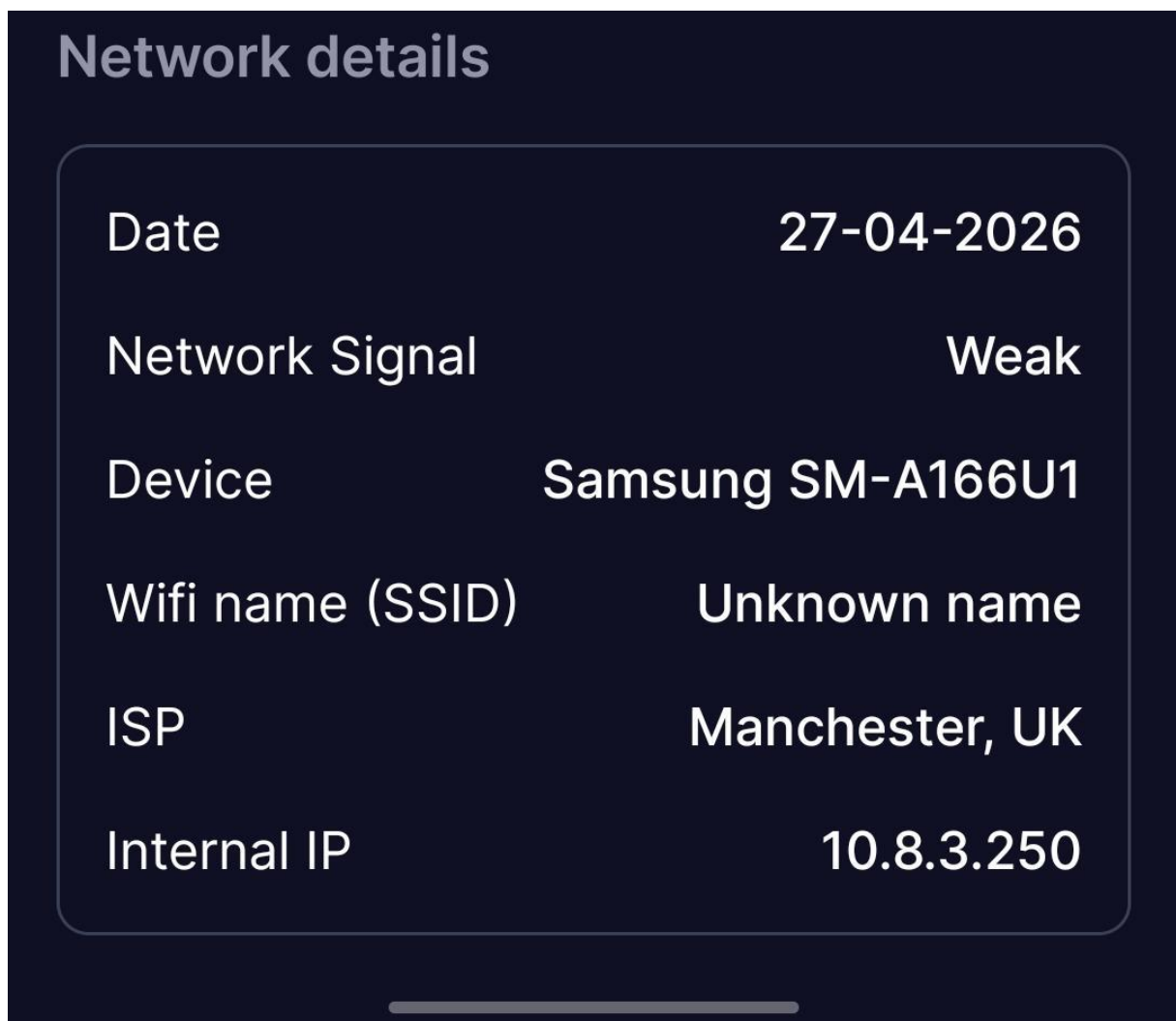
- Their satisfaction levels
- Common network issues (slow internet, dropped calls)

- Frequency of poor service experiences
- Network reliability
- Internet speed satisfaction
- Frequency of service disruptions
- Overall user experience

Process:

We ask people to download, run speed test app, and send their result to us for analysis.

This is a sample of what we received from users.



The image shows a dark-themed mobile app interface with a title 'Network details' at the top. Below the title, there is a rounded rectangle containing several data points, each with a label on the left and a value on the right. The data points are: Date (27-04-2026), Network Signal (Weak), Device (Samsung SM-A166U1), Wifi name (SSID) (Unknown name), ISP (Manchester, UK), and Internal IP (10.8.3.250). At the bottom of the screen, there is a horizontal line representing a mobile home indicator bar.

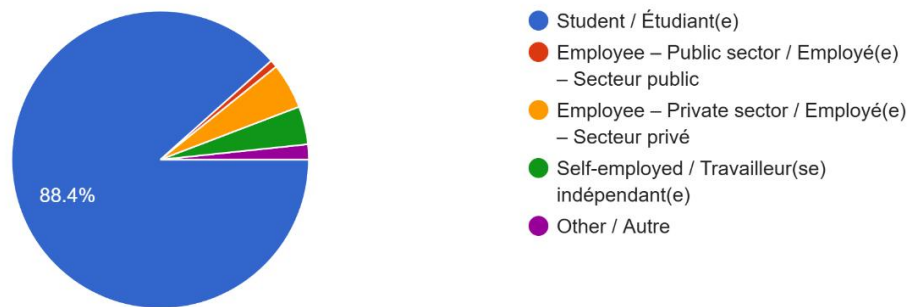
Date	27-04-2026
Network Signal	Weak
Device	Samsung SM-A166U1
Wifi name (SSID)	Unknown name
ISP	Manchester, UK
Internal IP	10.8.3.250

We shared the link to our Google form (containing useful questions) to people in the country and we obtained the following results.

https://docs.google.com/spreadsheets/d/110fy0ZcMNtOjLP353Mb6jg3b5rd3wSPkUIoJRf_X0VA/edit?usp=sharing

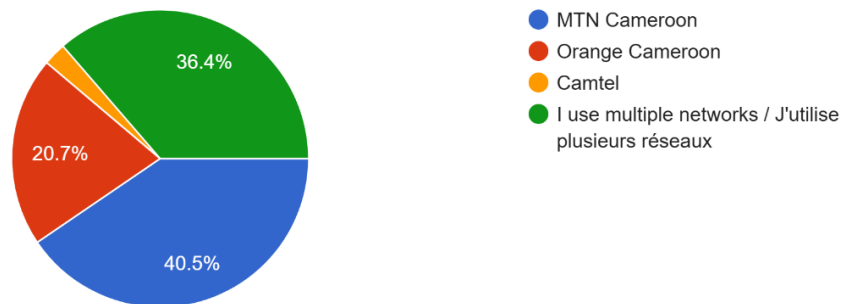
2. What is your occupation? /Quelle est votre profession ?

121 responses



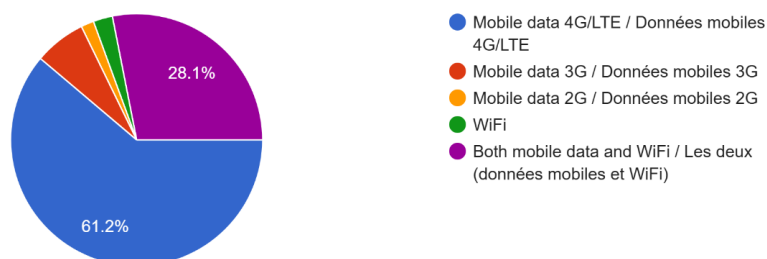
4. Which mobile network operator do you use? / Quel opérateur de réseau mobile utilisez-vous ?

121 responses



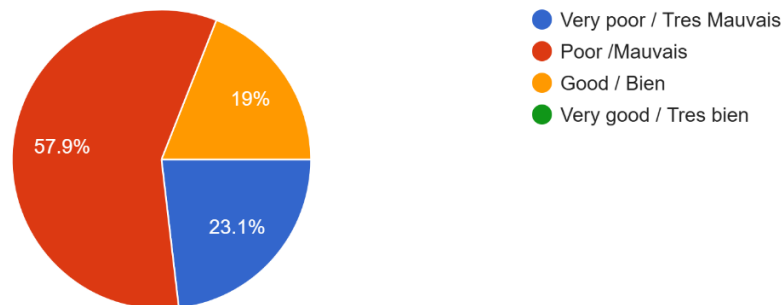
5. What type of internet connection do you mostly use? / Quel type de connexion internet utilisez-vous le plus souvent ?

121 responses



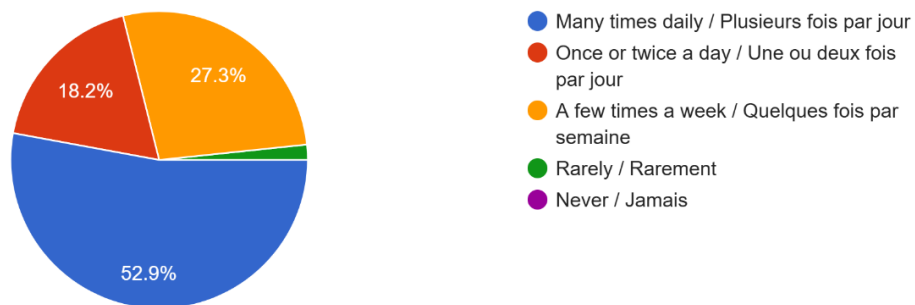
7. How would you rate your overall mobile network experience in Cameroon? / Comment évaluez-vous votre expérience globale du réseau mobile au Cameroun ?

121 responses



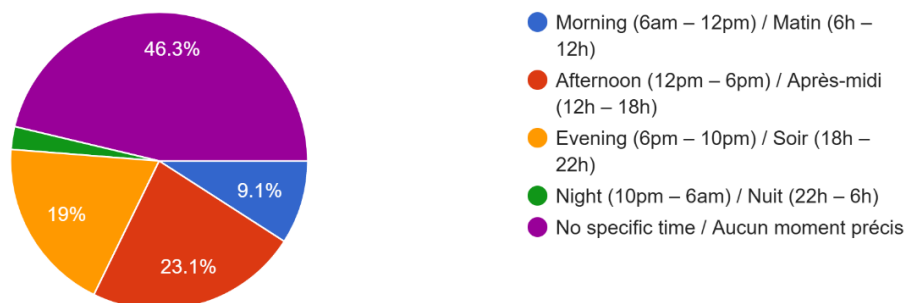
8. How often do you experience poor network quality? / À quelle fréquence rencontrez-vous une mauvaise qualité de réseau ?

121 responses



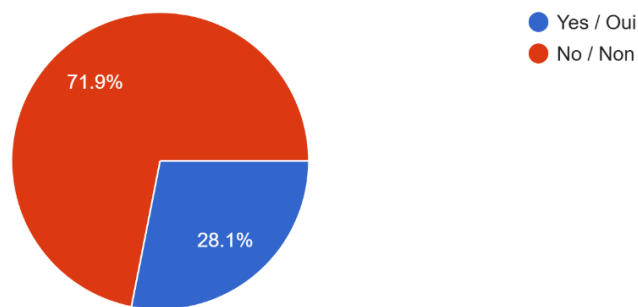
9. At what times do you mostly experience poor network quality? / À quels moments rencontrez-vous le plus souvent une mauvaise qualité de réseau ?

121 responses



13. Have you ever used a network monitoring app before? / Avez-vous déjà utilisé une application de surveillance réseau auparavant ?

121 responses



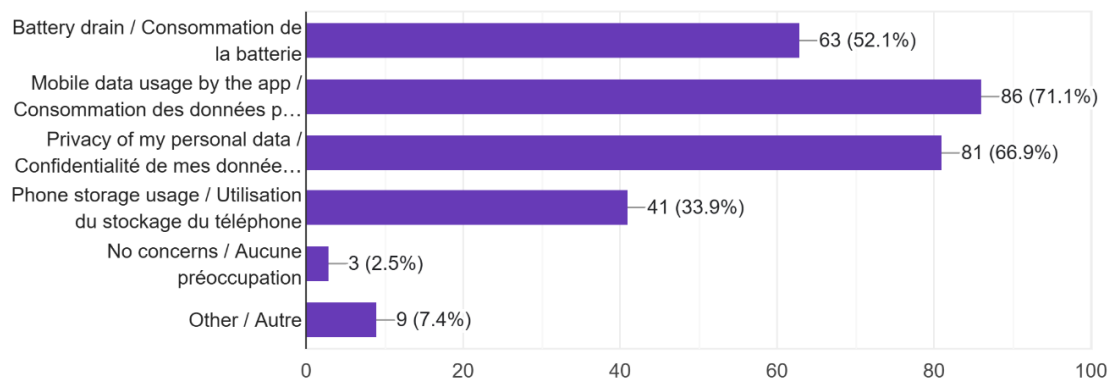
14. Would you be willing to install an app that runs in the background, collects your network quality data, and occasionally asks you to rate your experience? / Êtes-vous prêt à installer une application qui fonctionne en arrière-plan, collecte vos données de qualité réseau et vous demande occasionnellement d'évaluer votre expérience ?

121 responses



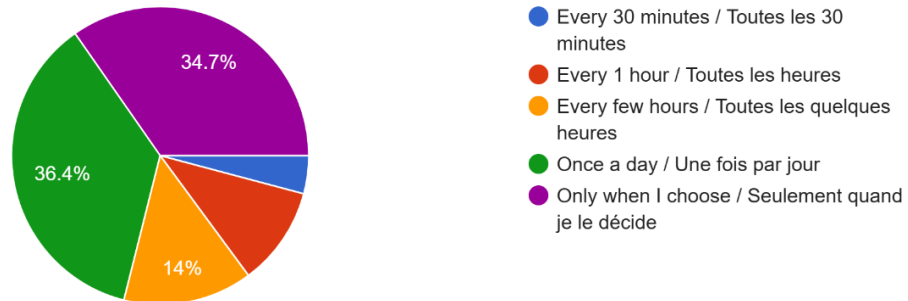
15. What concerns would you have about using such an app? (Select all that apply) / Quelles seraient vos préoccupations concernant l'utilisation de cette application ? (Sélectionnez tout ce qui s'applique)

121 responses



16. How often would you be comfortable being prompted to rate your network experience? / À quelle fréquence seriez-vous à l'aise pour être invité(e) à évaluer votre expérience réseau ?

121 responses



b. Interviews

We conducted informal interviews with users to gain deeper insights into their expectations and frustrations.

c. Brainstorming

As a team, we brainstormed features such as background data collection & monitoring, real-time alerts, and User rating system.

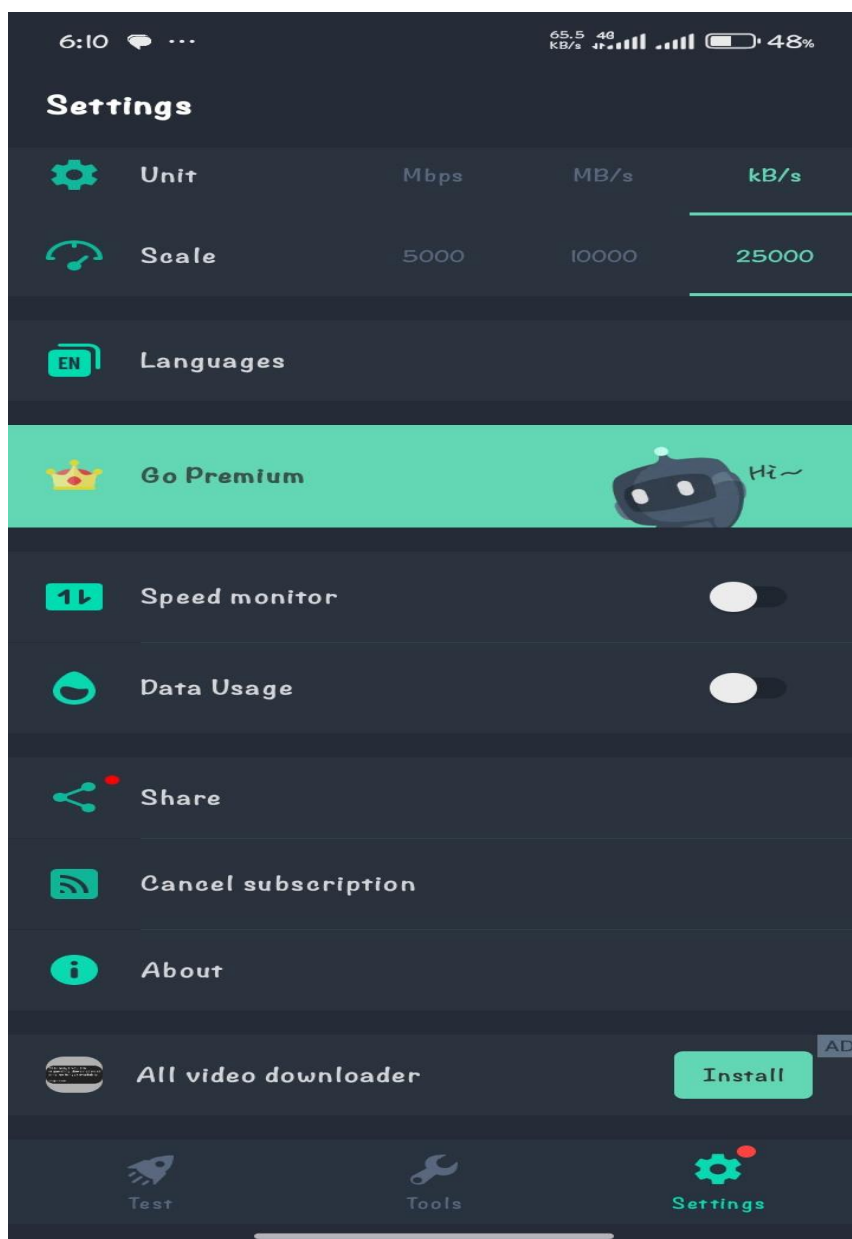
d. Reverse Engineering

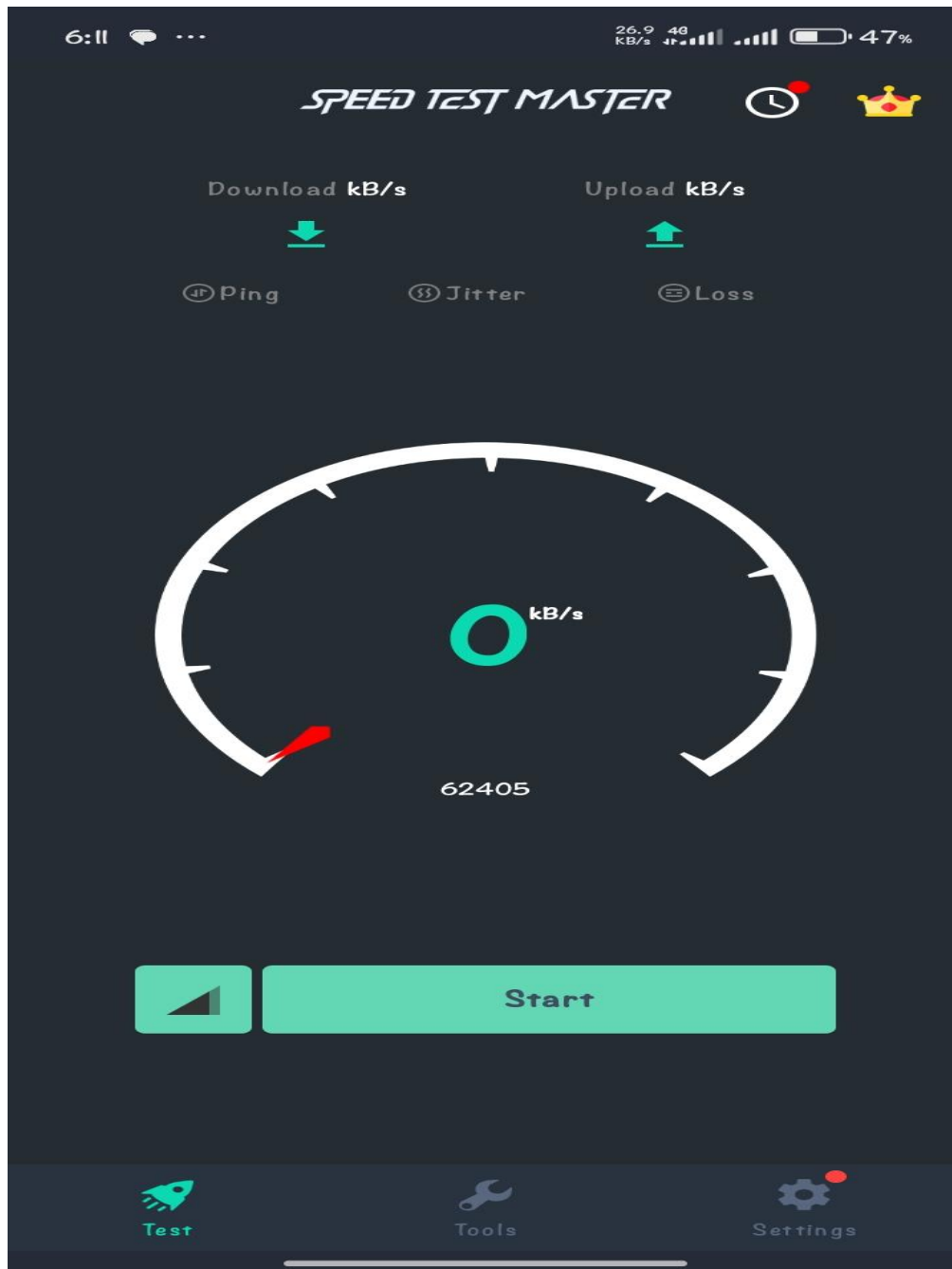
We analyzed existing applications such as:

- **Speedtest by Ookla**
- **OpenSignal**
- **NetMonster**

These platforms helped us understand how network metrics are collected and presented, best practices and limitations.

Here we have some features of the speed test app:





4. Data Gathering

Data gathering was carried out using two main approaches:

a. Survey Data

We collected responses from users regarding:

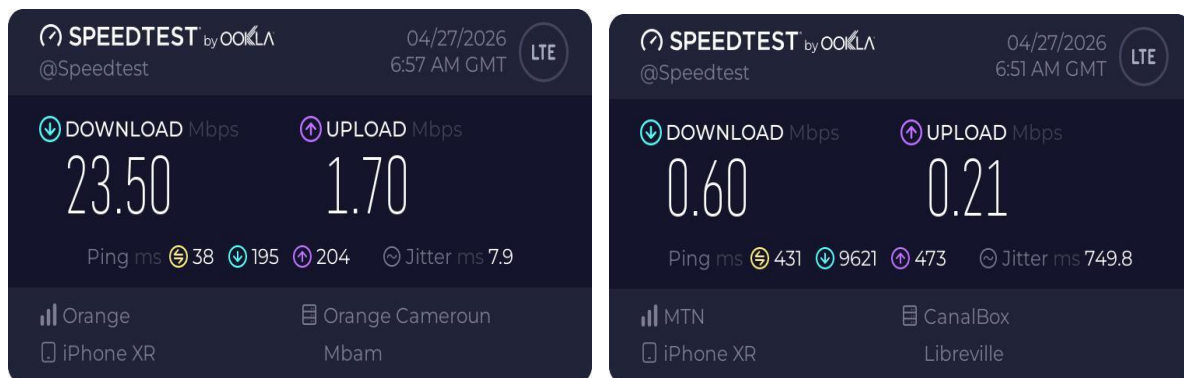
- Network satisfaction
- Common issues experienced (slow speed, call drops)

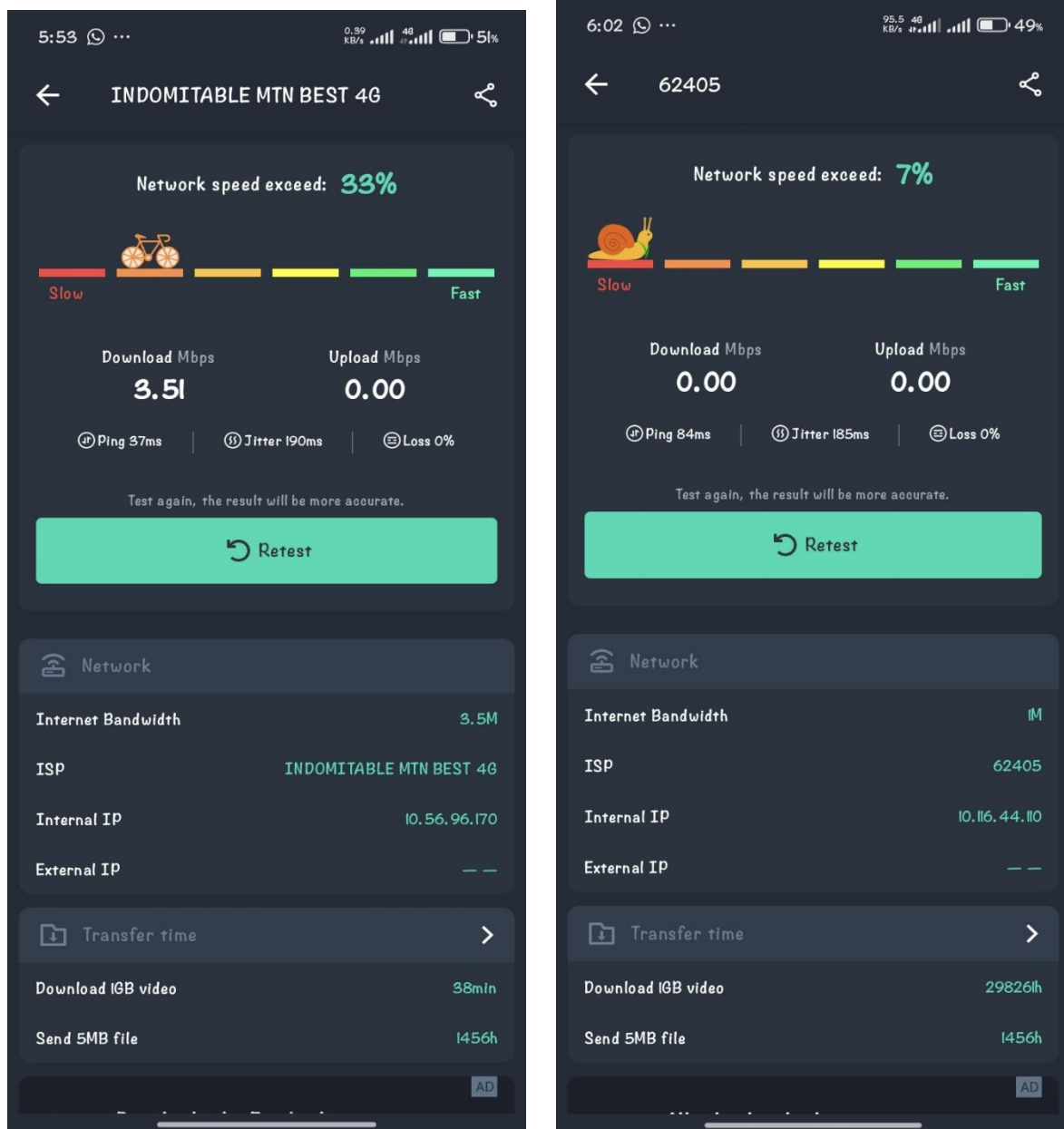
- Preferred features in a mobile app

b. Objective Data (Speedtest App Results)

We used results from the **Speedtest application** to collect:

- Download and upload speeds
- Latency (ping)
- Network type (3G, 4G, etc.)





This allowed us to combine **subjective user feedback** with **objective performance metrics**.

5. Data Cleaning

After collecting the data, we performed data cleaning to ensure accuracy and reliability:

- Removed duplicate survey responses
- Corrected inconsistent entries (e.g., different formats of network types)
- Handled missing values by filtering incomplete responses
- Standardized data formats (dates, numeric values)

This step is essential to ensure that analysis and system design are based on valid data.

From the duplicated, not standardized, inconsistent and incomplete responses of the users:

1	2	3	4	5	6
1. What is your age range? / Quelle est votre	2. What is your occupation? / Quelle est votre	3. Which city/town do you currently live in? / I	4. Which mobile network operator do you use	5. What type of internet connection do you m	6. What do you mainly use mobil
18 - 25	Student / Étudiant(e)	Buea	I use multiple networks / J'utilise plusieurs résea	Both mobile data and WIFI / Les deux (données m	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Bafoussam	Orange Cameroon	Both mobile data and WIFI / Les deux (données m	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Buea	MTN Cameroon	Mobile data 4G/LTE / Données mobiles 4G/LTE	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Buea	Orange Cameroon	Mobile data 4G/LTE / Données mobiles 4G/LTE	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Buea	I use multiple networks / J'utilise plusieurs résea	Mobile data 4G/LTE / Données mobiles 4G/LTE	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Buea	I use multiple networks / J'utilise plusieurs résea	Both mobile data and WIFI / Les deux (données m	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Bafoussam	Orange Cameroon	Mobile data 4G/LTE / Données mobiles 4G/LTE	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Niyi	MTN Cameroon	Mobile data 4G/LTE / Données mobiles 4G/LTE	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Douala	I use multiple networks / J'utilise plusieurs résea	Both mobile data and WIFI / Les deux (données m	Social media / Réseaux sociaux (
18 - 25	Self-employed / Travailleur(se) indépendant(e)	Bafoussam	Orange Cameroon	Mobile data 3G / Données mobiles 3G	Browsing, research / Navigatio
18 - 25	Student / Étudiant(e)	Buea	MTN Cameroon	Mobile data 4G/LTE / Données mobiles 4G/LTE	Social media / Réseaux sociaux (
18 - 25	Student / Étudiant(e)	Buea	I use multiple networks / J'utilise plusieurs résea	WIFI	Social media / Réseaux sociaux (

To well defined, and cleaned responses:

1	2	3	4	5	6	7	8	9
Q1_Age_Ran	Q2_Occupation	Q3_City	Q4_Network_Operat	Q5_Connection_Type	Q6_Internet_Uses	Q7_Network_F	Q8_Poor_Quality_Freq	Q9_Poor_Quality_Tim
18 - 25	Student	Buea	I use multiple networks	Both mobile data and WIFI	Social media, Browsing, research	Poor	Many times daily	No specific time
18 - 25	Student	Bafoussam	Orange Cameroon	Both mobile data and WIFI	Social media, Browsing, research	Poor	A few times a week	Evening (6pm - 10pm)
18 - 25	Student	Buea	MTN Cameroon	Mobile data 4G/LTE	Social media, Online classes, Mobile banking, Browsing, research	Poor	Many times daily	No specific time
18 - 25	Student	Buea	Orange Cameroon	Mobile data 4G/LTE	Social media	Poor	Many times daily	Afternoon (12pm - 6pm)
18 - 25	Student	Buea	I use multiple networks	Mobile data 4G/LTE	Social media, Streaming videos, Netflix, Online classes, Browsing, research	Poor	A few times a week	Evening (6pm - 10pm)
18 - 25	Student	Buea	I use multiple networks	Both mobile data and WIFI	Social media	Very poor	Many times daily	No specific time
18 - 25	Student	Bafoussam	Orange Cameroon	Mobile data 4G/LTE	Social media, Browsing, research	Good	A few times a week	No specific time
18 - 25	Student	Unknown	MTN Cameroon	Mobile data 4G/LTE	Social media, Streaming videos, Netflix, Online classes, Browsing, research	Poor	Many times daily	Evening (6pm - 10pm)
18 - 25	Student	Douala	I use multiple networks	Both mobile data and WIFI	Social media, Streaming videos, Netflix, Browsing, research	Poor	A few times a week	No specific time
18 - 25	Self-employed	Bafoussam	Orange Cameroon	Mobile data 3G	Browsing, research	Poor	A few times a week	No specific time
18 - 25	Student	Buea	MTN Cameroon	Mobile data 4G/LTE	Social media	Poor	Many times daily	Evening (6pm - 10pm)
18 - 25	Student	Buea	I use multiple networks	WIFI	Social media, Streaming videos, Netflix, Online classes, Mobile banking, Browsing, research	Good	Once or twice a day	Evening (6pm - 10pm)
18 - 25	Student	Buea	I use multiple networks	Mobile data 4G/LTE	Other	Poor	A few times a week	Night (10pm - 6am)
18 - 25	Student	Bafoussam	Orange Cameroon	Mobile data 4G/LTE	Social media	Poor	Many times daily	No specific time
18 - 25	Student	Douala	Orange Cameroon	Mobile data 4G/LTE	Social media	Poor	Once or twice a day	Morning (6am - 12pm)
26 - 35	Other	Douala	MTN Cameroon	Mobile data 4G/LTE	Streaming videos, Netflix, Online classes, Browsing, research	Very poor	Many times daily	No specific time
18 - 25	Student	Yaoundé	MTN Cameroon	Both mobile data and WIFI	Social media	Good	A few times a week	Afternoon (12pm - 6pm)
18 - 25	Student	Buea	MTN Cameroon	Mobile data 4G/LTE	Social media, Online classes, Browsing, research	Poor	Once or twice a day	No specific time
18 - 25	Student	Buea	I use multiple networks	Both mobile data and WIFI	Browsing, research	Poor	Many times daily	No specific time

We extracted useful data from the the pictures of the speed test app we received from users and classified them for better analysis and visualization.

B	C	D	E	F	G	H	I
Device type	User ID	Latency(ms)	DL(Mbps)	UL(Mbps)	Bandwidth (Mbps)	Jitter (ms)	Packet Loss
android	690799180	158	16,2	1,86	16,2	121	0
ios	672542768	132	3	3	2.4	198	8
ios	658487098	79	1	1	0.7	79	15
android	620112911	64	9.29	1.79	9.3	112	0
ios	659206984	367	1	1	0.2	--	23
android	697029633	252.1	0.79	3.7	2.5	7.62	0
ios	620173294	143	0.42	0.45	1	48	0
android	699362055	2845	4	2	12.4		0
ios	694067501	2536	13	2	12.6		0
android	675080477	140	1.29	0.35	1.3	19	0

6. Observations from Collected Data

From the analysis of survey responses and Speedtest results, we observed that:

- A large percentage of users reported **low satisfaction with network speed**, especially during peak hours.
- Many users experienced **frequent network instability**, including dropped connections and slow browsing.
- There is a noticeable gap between **perceived performance and actual measured speeds**, indicating inconsistency.
- Users expressed strong interest in a **simple app that provides feedback and monitors performance automatically with low battery and data consumption**.
- Speedtest results showed **variability in latency and bandwidth depending on location**, highlighting coverage issues.

These observations confirm the need for a system that continuously collects QoE data.

7. User Reluctance Assessment

In our project to design a mobile application for collecting user experience data from mobile network subscribers in Cameroon, assessing user reluctance was an essential part of the requirement gathering phase. User reluctance refers to the resistance potential users may have toward adopting or continuously using a future application. Since our proposed app will require background operation, location access, and periodic feedback, understanding these reluctance factors now helps us gather the right requirements before any implementation begins. We identified potential resistance from users regarding the use of the application:

Challenges identified:

Reluctance Factor	Description
Battery drain fear	Users may worry that continuous background operation will quickly deplete their phone battery
Data usage concern	Users on limited or expensive data plans may fear unexpected data consumption
Privacy invasion	Collection of location and device metrics could be perceived as surveillance
Annoyance from prompts	Frequent feedback requests could interrupt normal phone activities
Lack of personal benefit	Users may see no reason to help the operator without receiving something in return

Low trust in operators	Mobile network operators are not always trusted by subscribers in Cameroon
------------------------	--

Solutions:

- Implement clear privacy policies
- Allow users to control data sharing
- Optimize the background processing of the app for low battery usage
- Provide transparency on how data is used
- Clear explanation of benefits

III. Requirement Analysis

2.1 Review and Analysis of Gathered Requirements

During the requirement gathering phase, we collected information from mobile subscribers through surveys and observations. We also analyzed objective network metrics obtained using the Speedtest application. Existing platforms such as OpenSignal and NetMonster were reviewed to understand how similar systems operate.

The gathered requirements were analyzed based on the following factors:

- **Completeness**

We verified whether the collected requirements fully addressed user needs and project objectives. Users consistently requested features such as automatic network monitoring, background execution, battery efficiency, and real-time feedback collection.

- **Clarity**

Requirements were rewritten in simple and understandable language to ensure that both technical and non-technical stakeholders could understand them easily.

- **Technical Feasibility**

We evaluated whether the requested functionalities could be implemented using Android APIs and available technologies. Features such as signal strength monitoring, latency measurement, location tracking, and background execution were confirmed to be technically feasible.

- **Dependency Relationships**

Some requirements depend on others. For example:

- Real-time analytics depend on continuous data collection.
- QoE reporting depends on successful storage of network metrics and user feedback.
- User notifications depend on background services running properly.

2.2 Identification of Inconsistencies and Missing Information

After reviewing the gathered data, some inconsistencies and missing information were identified.

- **Inconsistencies**

Some users reported acceptable browsing experiences even when Speedtest results indicated low bandwidth or high latency. This showed that user perception may differ from actual network measurements.

- **Missing Information**

Certain requirements were not initially clear, including:

- Preferred frequency for feedback notifications
- Acceptable battery consumption levels
- Offline data synchronization requirements

- Device compatibility requirements

These issues were clarified through additional analysis and follow-up discussions.

2.3 Requirement Prioritization

Requirements were prioritized according to importance, feasibility, and impact on system objectives.

High Priority Requirements:

- Continuous background monitoring
- Automatic collection of network metrics
- User satisfaction feedback collection
- Secure storage of collected data
- Timestamp and GPS location recording

Medium Priority Requirements:

- Data visualization dashboard
- Offline data synchronization
- Notification and alert system

Low Priority Requirements:

- Social sharing functionality
- Multi-language support
- Personalized themes

Prioritization helps developers focus on the most critical functionalities first.

2.4 Requirement Classification

The system requirements were classified into functional and non-functional requirements.

Functional Requirements:

- The application must collect network metrics automatically.
- The application must allow users to submit feedback and satisfaction ratings.
- The application must store timestamps and location data.
- The application must synchronize collected data with the server.
- The application must generate QoE reports for analysis.

Non-Functional Requirements:

- The system must be secure and protect user privacy.
- The application must consume minimal battery power.
- The system must provide high reliability and availability.
- The application must be user-friendly and easy to use.
- The system must support scalability for large numbers of users.

2.5 Validation of Requirements with Stakeholders

The identified requirements were validated with stakeholders to ensure that they align with actual user needs and business objectives.

Validation involved:

- Reviewing survey results with team members
- Comparing findings with existing platforms
- Evaluating requirements against project objectives
- Considering the expectations of mobile network operators such as MTN Cameroon, Orange Cameroon, and Camtel

Stakeholders confirmed that:

- Real-time QoE monitoring is necessary
- User privacy must be prioritized
- Automated data collection is more reliable than manual reporting
- Lightweight applications are preferred by users

3. Software Requirements Specification (SRS)

The Software Requirements Specification (SRS) document defines the complete behavior and functionality of the proposed QoE mobile application. It serves as a communication tool between developers, stakeholders, testers, and project managers. The SRS ensures that all parties clearly understand the system objectives, requirements, constraints, and expected outcomes.

3.1 Purpose of the System

The purpose of the system is to develop a mobile application capable of collecting, monitoring, storing, and analyzing Quality of Experience (QoE) data from mobile network subscribers in real time.

The application combines:

- Subjective feedback from users
- Objective network performance metrics

The collected data will support:

- Network optimization
- Service quality evaluation
- Customer satisfaction improvement
- Decision-making by network operators

3.2 Scope of the System

The system will operate on Android mobile devices and perform the following functions:

- Run continuously in the background

- Collect network metrics automatically
- Prompt users periodically for feedback
- Store and transmit data securely
- Generate QoE analytics and reports

The application focuses on mobile network performance monitoring within Cameroon.

3.3 Stakeholders and Users

Primary Users:

- Mobile subscribers
- Network analysts
- System administrators

Secondary Stakeholders:

- Mobile network operators
- Researchers
- Regulatory authorities

Each stakeholder interacts differently with the system:

- Subscribers provide feedback
- Analysts review reports
- Operators use data for optimization

3.4 Functional Requirements

1. User Authentication

- The system must allow secure user login and registration.

2. Background Monitoring

- The application must continuously collect network metrics while running in the background.

3. Feedback Collection

- Users must be able to rate their network experience.

4. Network Metrics Collection

The application must collect:

- Signal strength
- Network type
- Download speed
- Upload speed
- Latency
- Packet loss
- Jitter
- Timestamp
- GPS location

5. Data Synchronization

- The application must send collected data to a backend server.

6. Report Generation

- The system must generate reports and analytics dashboards.

3.5 Non-Functional Requirements

1. Performance

- The application must respond quickly and operate efficiently.

2. Security

- User data must be encrypted and protected.

3. Reliability

- The system must operate continuously without crashes.

4. Usability

- The interface must be simple and intuitive.

5. Scalability

- The system must support large numbers of users.

6. Maintainability

- The system should be easy to update and maintain.

3.6 System Features

Main features of the application include:

- Real-time QoE monitoring
- Background execution
- User feedback forms
- Speedtest integration
- Interactive dashboard
- Notification system
- Analytics and visualization tools

3.7 Hardware and Software Requirements

Hardware Requirements:

- Android smartphone
- GPS-enabled device
- Internet connectivity

Software Requirements:

- Android Studio
- Java/Kotlin
- Firebase/MySQL

- REST APIs
- Node.js backend

3.8 Constraints and Assumptions

Constraints:

- Internet access is required for synchronization.
- Android permissions are necessary for location and network monitoring.

Assumptions:

- Users will allow background execution.
- Mobile operators will benefit from collected QoE data.
- Users will provide honest feedback.

3.9 Security Requirements

To protect user information:

- Data transmission must use encryption.
- Authentication mechanisms must be implemented.
- Personal information must remain confidential.
- Access to reports must be restricted to authorized users only.

IV. System Modelling and Design

In order to successfully design such a system, **system modeling and design techniques** are essential. System modeling provides a structured way to represent the application's architecture, processes, and interactions before actual implementation. It helps identify requirements, visualize workflows, and ensure that all stakeholders share a common understanding of how the system will function.

For this project, six key diagrams are used to capture different perspectives of the system

Which are:

1. Context Diagram

1.1. Introduction

A context diagram is a high-level system model that illustrates the boundaries of a system and its interactions with external entities. It provides an overall view of how the system communicates with users, external systems, and stakeholders without exposing internal implementation details.

This report presents and explains the context diagram for the Mobile Quality of Experience (QoE) Data Collection System, a mobile application designed to collect, store, and analyze both subjective and objective network experience data from mobile network subscribers in Cameroon.

1.2. Purpose of the Context Diagram

The purpose of this context diagram is to:

- Define the system boundary of the QoE Mobile App System.
- Identify all external actors and systems interacting with the application.
- Illustrate the flow of data between the system and its environment.
- Provide stakeholders with a clear and simple understanding of system interactions.

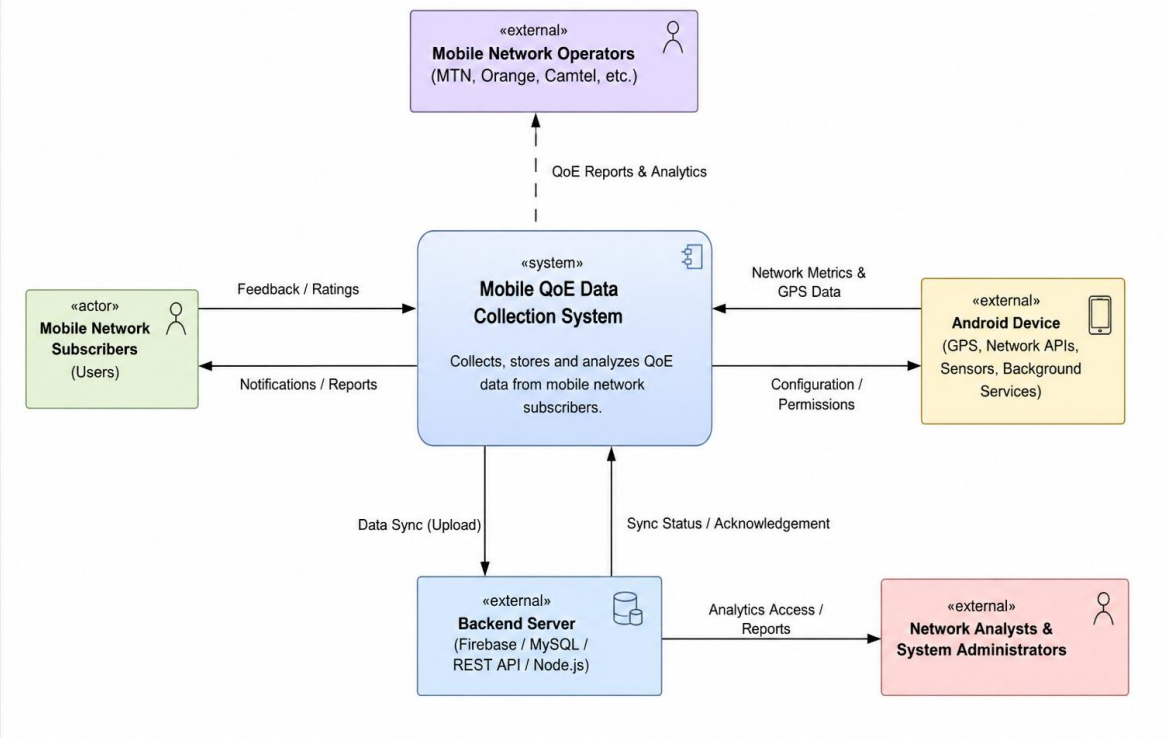
This diagram serves as a foundation for further system modeling, including use case diagrams, sequence diagrams, and data flow diagrams.

1.3. Overview of the System

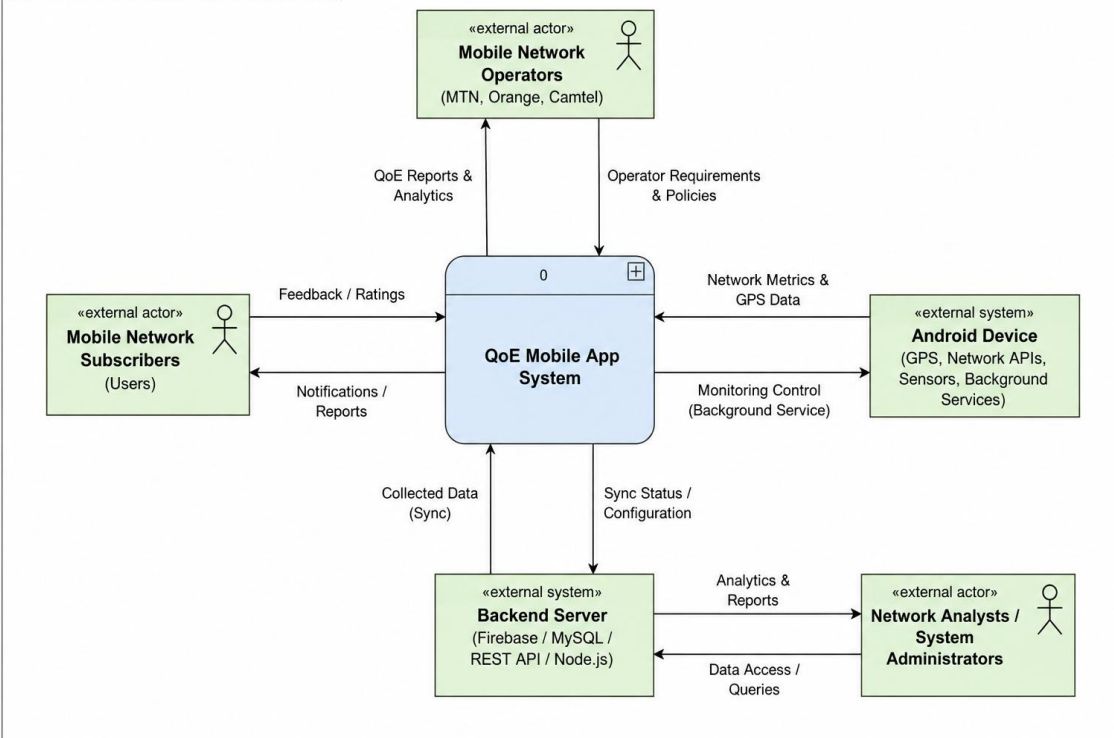
The QoE Mobile App System is the central system represented in the context diagram. It is responsible for:

- Collecting network performance metrics automatically.
- Gathering user satisfaction feedback and ratings.
- Storing and synchronizing data with a backend server.
- Generating QoE analytics and reports.

req [Context Diagram] Mobile QoE Data Collection System [Context Diagram]



pkg Context Diagram [QoE Mobile App System]



1.4. External Entities and Their Roles

1.4.1. Mobile Network Subscribers (Users)

Mobile Network Subscribers are the primary users of the system. They interact with the QoE Mobile App by:

- Providing feedback and satisfaction ratings
- Allowing background monitoring of network performance
- Receiving notifications and summarized reports

Their input represents the subjective Quality of Experience (QoE), which complements objective network measurements.

1.4.2. Android Device

The Android Device represents the mobile platform and hardware environment on which the application runs. It provides:

- Access to GPS location data.
- Network and telephony APIs.
- Signal strength and network type information.
- Background services required for continuous monitoring.

The QoE Mobile App relies on these system resources to collect accurate and real-time network metrics.

1.4.3. Backend Server

The Backend Server is an external system responsible for data storage and processing.

- Receiving synchronized data from the mobile application.
- Storing large volumes of QoE data securely.
- Supporting analytics, reporting, and dashboard services.
- Managing authentication and access control.

1.4.4. Network Analysts and System Administrators

Network Analysts and System Administrators access the system indirectly through the backend server. Their responsibilities include:

- Viewing QoE reports and analytics dashboards.
- Analyzing network performance trends.
- Managing system configurations and user access.
- Supporting decision-making for network optimization.

1.4.5. Mobile Network Operators

Mobile Network Operators such as MTN Cameroon, Orange Cameroon, and Camtel are key stakeholders. They:

- Receive aggregated QoE reports and analytics.
- Use insights to improve network coverage and quality.

- Provide policies and operational requirements that influence system design.

Although they do not interact directly with mobile application, they are major beneficiaries of the collected data.

1.5. Data Flow Description

- The context diagram illustrates several key data flows:
- **Feedback / Ratings:** From Mobile Network Subscribers to the QoE Mobile App.
- **Notifications / Reports:** From the QoE Mobile App to Subscribers.
- **Network Metrics & GPS Data:** From the Android Device to the QoE Mobile App.
- **Collected Data (Synchronization):** From the QoE Mobile App to the Backend Server.
- **Analytics & Reports:** From the Backend Server to Network Analysts and Administrators.
- **QoE Reports & Analytics:** From the QoE Mobile App System to Mobile Network Operators.

These flows show how subjective and objective data are collected, processed, and utilized by different stakeholders.

1.6. System Boundary Definition

The context diagram clearly defines the system boundary by placing the QoE Mobile App System at the center. All other entities lie outside this boundary and interact with the system through well-defined interfaces.

This separation ensures:

- Better understanding of system responsibilities.
- Clear distinction between internal processing and external interactions.
- Improved system security and maintainability.

1.7. Conclusion

The context diagram provides a comprehensive high-level representation of the Mobile QoE Data Collection System and its interaction with external actors and systems. It highlights the central role of the mobile application in collecting user experience data, the importance of backend services for storage and analysis, and the value of the system to network operators and analysts.

This diagram establishes a solid conceptual foundation for detailed system design and supports effective communication between developers, stakeholders, and project evaluators.

2. Data Flow Diagram

2.1. Introduction to Data Flow Diagrams

A Data Flow Diagram (DFD) is a structured analysis tool that shows how data moves through a system. It uses:

- Processes (functions that transform data)
- External Entities (actors outside the system)
- Data Stores (repositories of data)
- Data Flows (arrows showing direction and labels of data movement)

The QoE System is designed to collect, process, and analyze user feedback and network metrics to generate Quality of Experience reports for multiple stakeholders.

DFDs are drawn at different levels. In this project, only Level 0 (Context) and Level 1 (System Processes) were implemented.

Tools used : Gane & Sarson notation in Draw.io

2.2. Level 0 :Context Diagram

✓ **Central Process**

- QoE System

✓ **External Entities & Data Flows**

User

- *Submit Feedback, Request Metrics* --> QoE System

- *Received/View Metrics, Notification* <-- QoE System

Mobile Operator

- *Request Data* --> QoE System

- *QoE Reports* <-- QoE System

Researcher

- *Request Data* --> QoE System

- *QoE Reports* <-- QoE System

2.2.1. Purpose

Defines the system boundary and shows how external actors exchange feedback, metrics, and reports with the QoE System.

2.3. Level 1: System Processes

Processes

- User Interaction / Interface

- Network Monitoring

- Feedback Collection

- QoE Report Analysis

Data Stores

- Metrics Data

- Feedback Data

- QoE Report Data

Data Flows

- User --> User Interaction / Interface: *Submit Feedback, Request Metrics*

- QoE System --> User: *Received/View Metrics, Notification*

- Mobile Operator --> Network Monitoring: *Request*

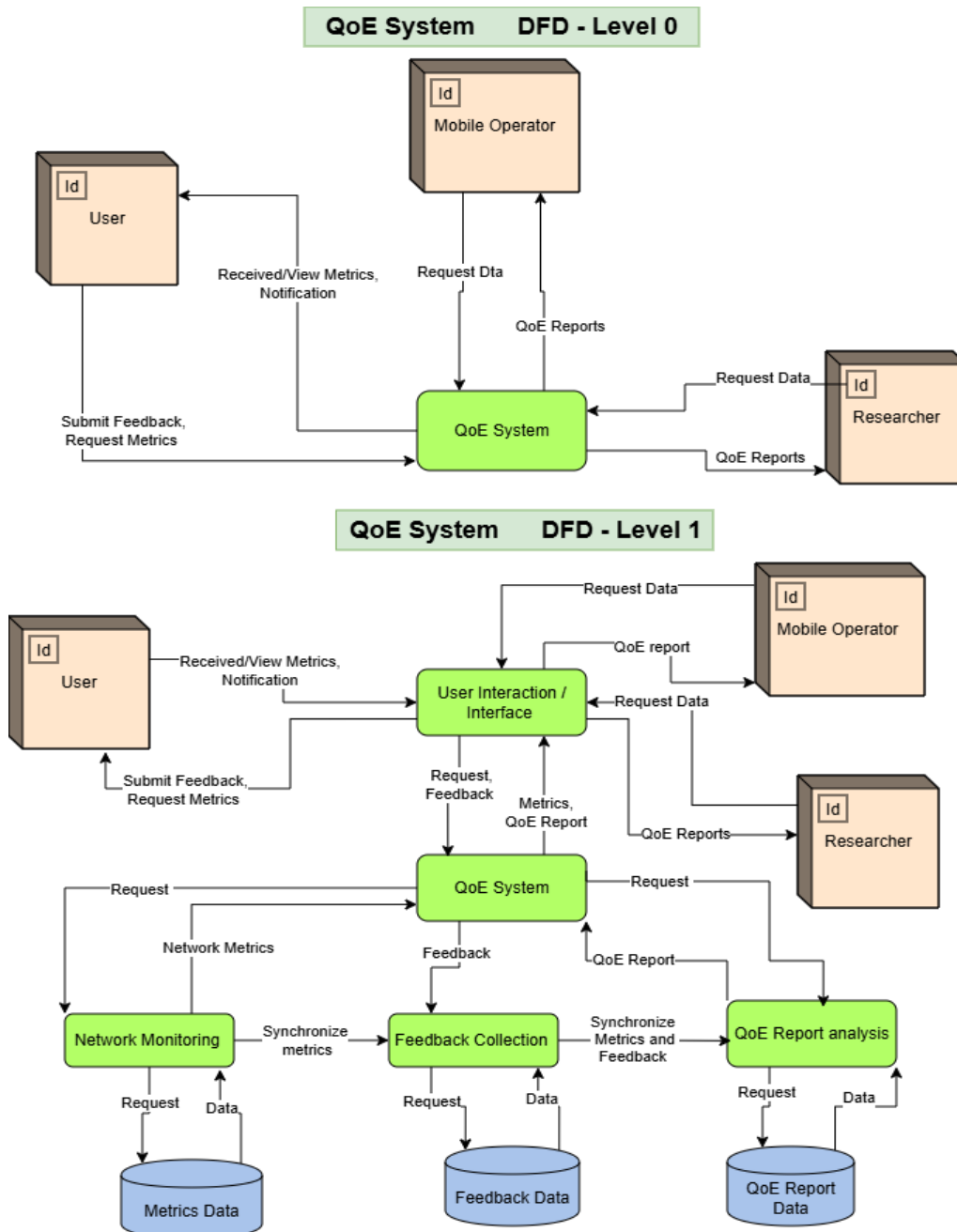
- Researcher --> Feedback Collection: *Request Data*
- QoE Report Analysis --> Mobile Operator & Researcher: *QoE Report*
- Network Monitoring --> Feedback Collection: *Synchronize Metrics and Feedback*
- Network Monitoring --> Metrics Data: *Network Metrics*
- Feedback Collection --> Feedback Data: *Feedback*
- QoE Report Analysis --> QoE Report Data: *Metrics, QoE Report*

Purpose

Shows the internal subsystems of the QoE System, their interactions, and how data is stored and synchronized to produce reports.

2.4. Conclusion

- The QoE System DFDs (Levels 0 and 1) provide a clear view of:
 - How users, operators, and researchers interact with the system.
 - How feedback and metrics are collected and synchronized.
 - How reports are generated and distributed.



3. USE CASE DIAGRAM

3.1. Introduction

A Use Case Diagram is a UML diagram used to show how users interact with a system. It identifies the actors involved and the services or functions provided by the system. For this project, the use case diagram represents how mobile subscribers, network operators, and the backend server interact with the mobile application designed to collect Quality of Experience (QoE) data from users.

3.2. Purpose of the Use Case Diagram

The purpose of this diagram is to show interactions between users and the system, identify the main functionalities of the application, define the responsibilities of each actor, and provide a clear overview of system behavior.

3.3. Actors Identified

- **Mobile Subscriber:** Main user of the application who views network statistics, submits feedback, receives notifications, and manages settings.
- **Network Operator:** Accesses synchronized data to analyze network performance and user experience.
- **Backend System:** Stores, processes, and synchronizes collected network data.

3.4. Main Use Cases

- Background Monitoring
- View Real-Time Statistics
- Receive Notifications
- Collect Signal Strength
- Measure Latency & Ping
- Measure Download/Upload Speed
- GPS Location Tagging
- Offline Data Storage
- Backend Synchronization
- Provide QoE Feedback
- Submit Manual Reports
- Generate Weekly Reports
- Bilingual Support
- Manage Data Sharing

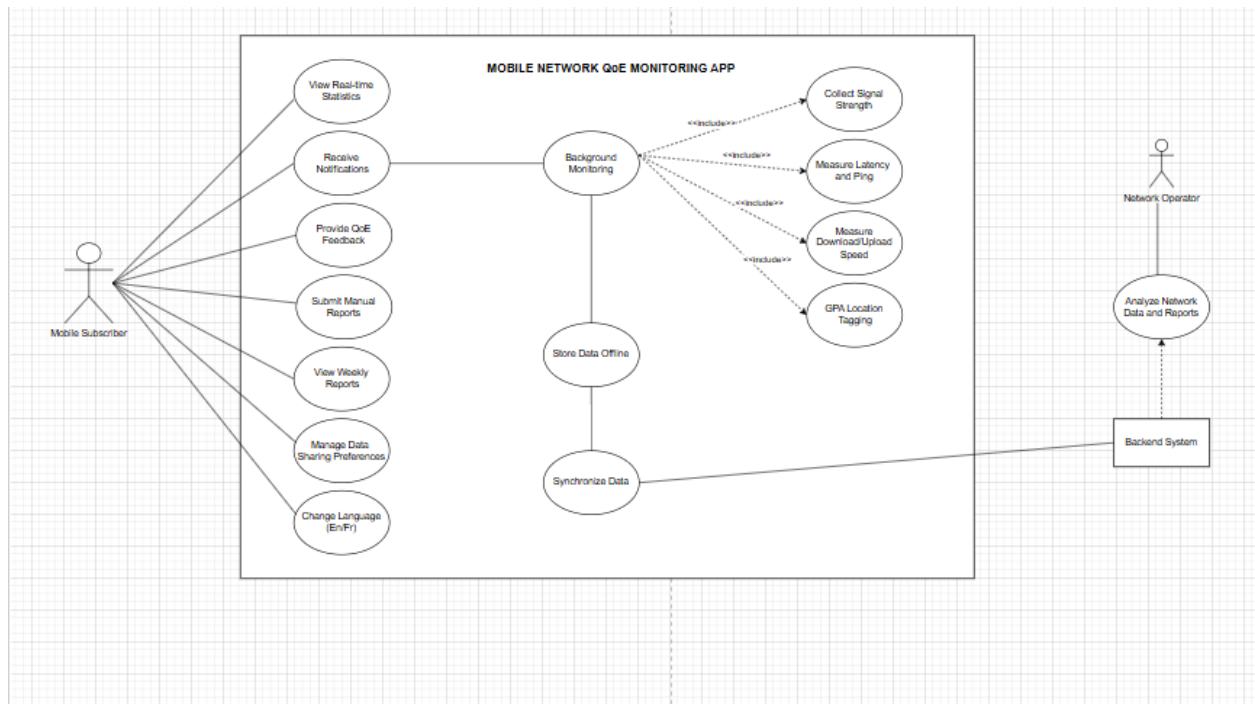
3.5. Relationships Between Use Cases

The diagram uses relationships where some use cases depend on others. For example, Background Monitoring includes Signal Strength Collection, Latency & Ping Measurement, Download/Upload Speed Measurement, and GPS Location Tagging.

3.6. Explanation of System Flow

The mobile subscriber interacts with the application to monitor network quality and provide feedback. The app continuously collects network metrics such as signal strength, speed, and latency. If internet connectivity is unavailable, data is stored locally and synchronized once connectivity is restored. The network operator then analyzes the synchronized data for performance evaluation.

3.7. Diagram



3.8. Conclusion

The use case diagram provides a high-level representation of system functionality and interactions. It helps stakeholders and developers understand the core services offered by the application and supports the design and implementation phases.

4. SEQUENCE DIAGRAM

4.1. Introduction

Sequence diagrams describe dynamic interactions between system components over time.

To complement the static view, we analyze the **dynamic behavior** of the system. In our project, we identified **five main scenarios** that are most relevant to QoE data collection and user interaction. Each scenario is represented by a sequence diagram that shows step-by-step interactions among components.

4.2. Scenarios

4.2.1. Scenario 1: App Launch and Background Monitoring Start

Scenario: The user opens the QoE application for the first time and the background service is initiated.

Actors/Components: Mobile Subscriber, Mobile App UI, Background Service, Device APIs, Local Database.

Flow Description:

The sequence begins when the Mobile Subscriber taps the app icon to open the application. The Mobile App UI responds by displaying a splash screen, then immediately requests permissions from the Device APIs, specifically GPS and network access permissions. If permissions are granted, the app proceeds to display the home dashboard. If permissions are denied, a permission request dialog is shown to the user (alternative flow).

Once the home dashboard is displayed, the Mobile App UI starts the Background Service. The Background Service then performs three initialization steps: it registers with Android WorkManager to ensure persistence, initializes the telephony listener, and initializes the GPS location listener. Once all

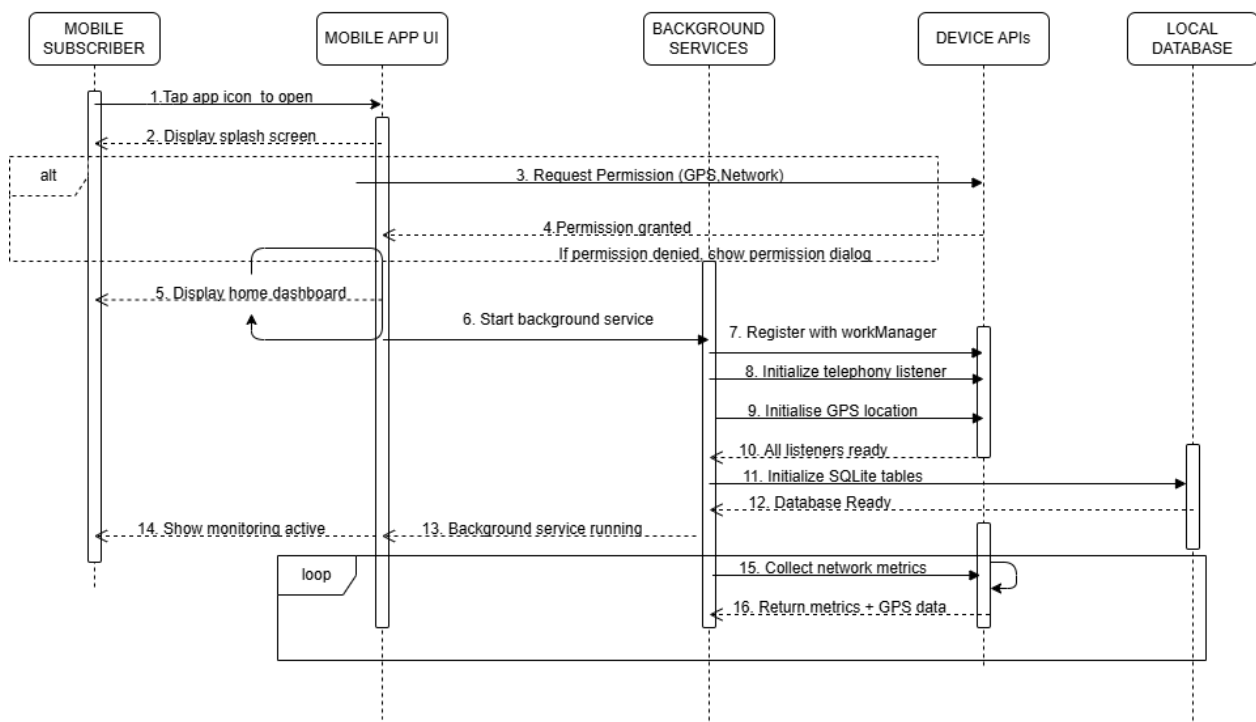
listeners are ready, the Background Service initializes the SQLite tables in the Local Database, which confirms that the database is ready.

The Background Service then reports back to the Mobile App UI that it is running, and the UI displays a "monitoring active" indicator to the user.

A loop begins, executing every 60 seconds while the app is running: the Background Service requests network metrics from the Device APIs, which return both metrics and GPS data.

Note: The background service continues running silently even when the user closes the app. Android Work Manager ensures persistence. The service respects Android Doze mode and battery optimization policies.

APP LAUNCH AND BACKGROUND MONITORING



4.2.2. Scenario 2 :Network Metrics Collection and Offline Storage

Scenario: The background service periodically collects network metrics, tags them with GPS coordinates, and stores them locally when the device is offline.

Actors/Components: Background Service, Telephony Manager API, GPS Location API, Network Manager API, Local SQLite Database.

Flow Description:

This diagram operates within a continuous loop that fires every 60 seconds as part of the periodic data collection cycle.

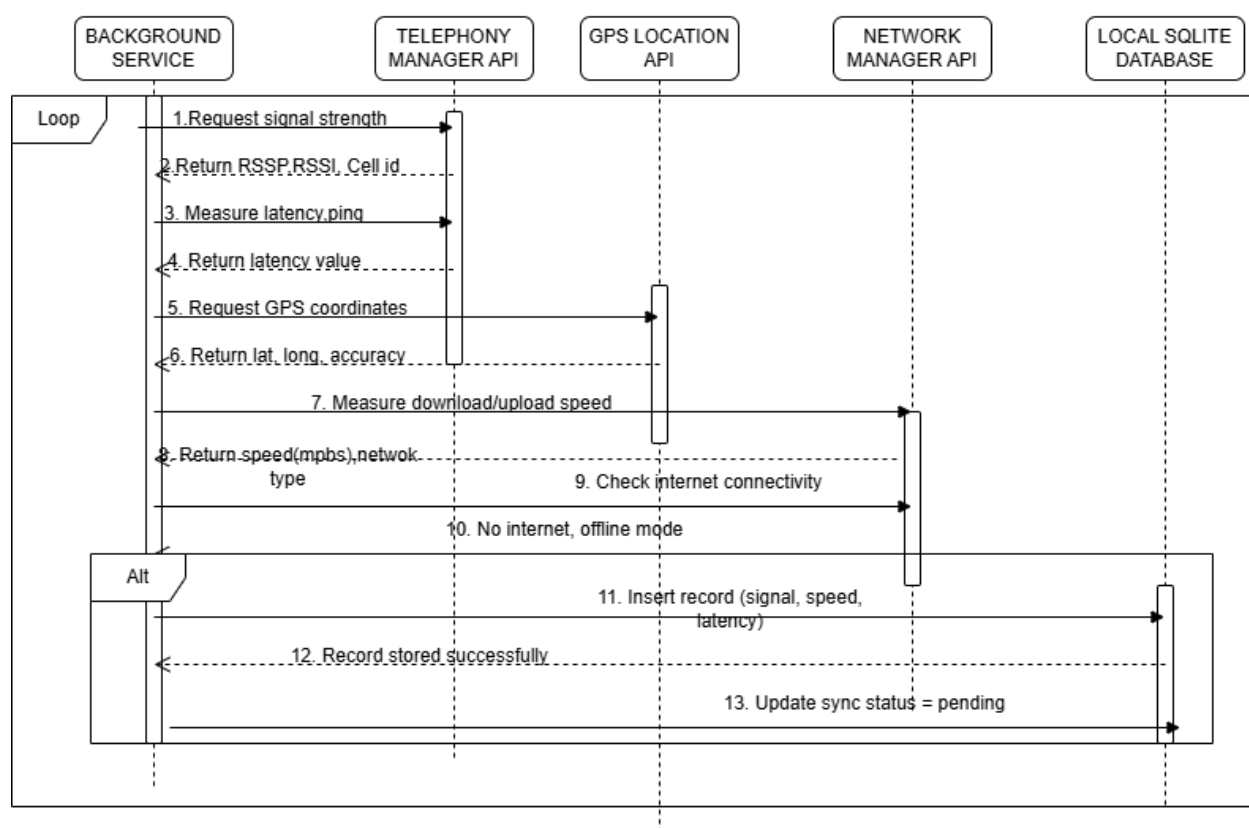
At the start of each cycle, the Background Service requests signal strength data (RSRP/RSSI) from the Telephony Manager API, which returns RSRP, RSSI, and cell ID values. The Background Service then measures latency via a ping request to the Network Manager API, which returns the latency value in milliseconds.

Next, the Background Service requests GPS coordinates from the GPS Location API, which returns latitude, longitude, and accuracy values. The Background Service then instructs the Network Manager API to measure download and upload speed, which returns speed in Mbps along with the network type.

The Background Service then checks internet connectivity via the Network Manager API. If there is no internet — meaning the device is in offline mode — an alternative flow is triggered: the Background Service inserts a record containing signal strength, speed, latency, GPS coordinates, and a timestamp into the Local SQLite Database. The database confirms the record is stored successfully with a pending sync flag set to true. The Background Service then updates the sync status to PENDING.

Note: Every record is tagged with GPS coordinates, timestamp, network operator name, and device model. Data is stored in SQLite with a pending sync flag — data is never lost even without connectivity.

NETWORK METRICS COLLECTION AND OFFLINE STORAGE



4.2.3. Scenario 3: Offline to Online Data Synchronization

Scenario: Internet connectivity is restored and the application syncs all pending local data to the backend server.

Actors/Components: Background Service, Network Manager API, Local SQLite Database, Encryption Module, Backend Server API.

Flow Description:

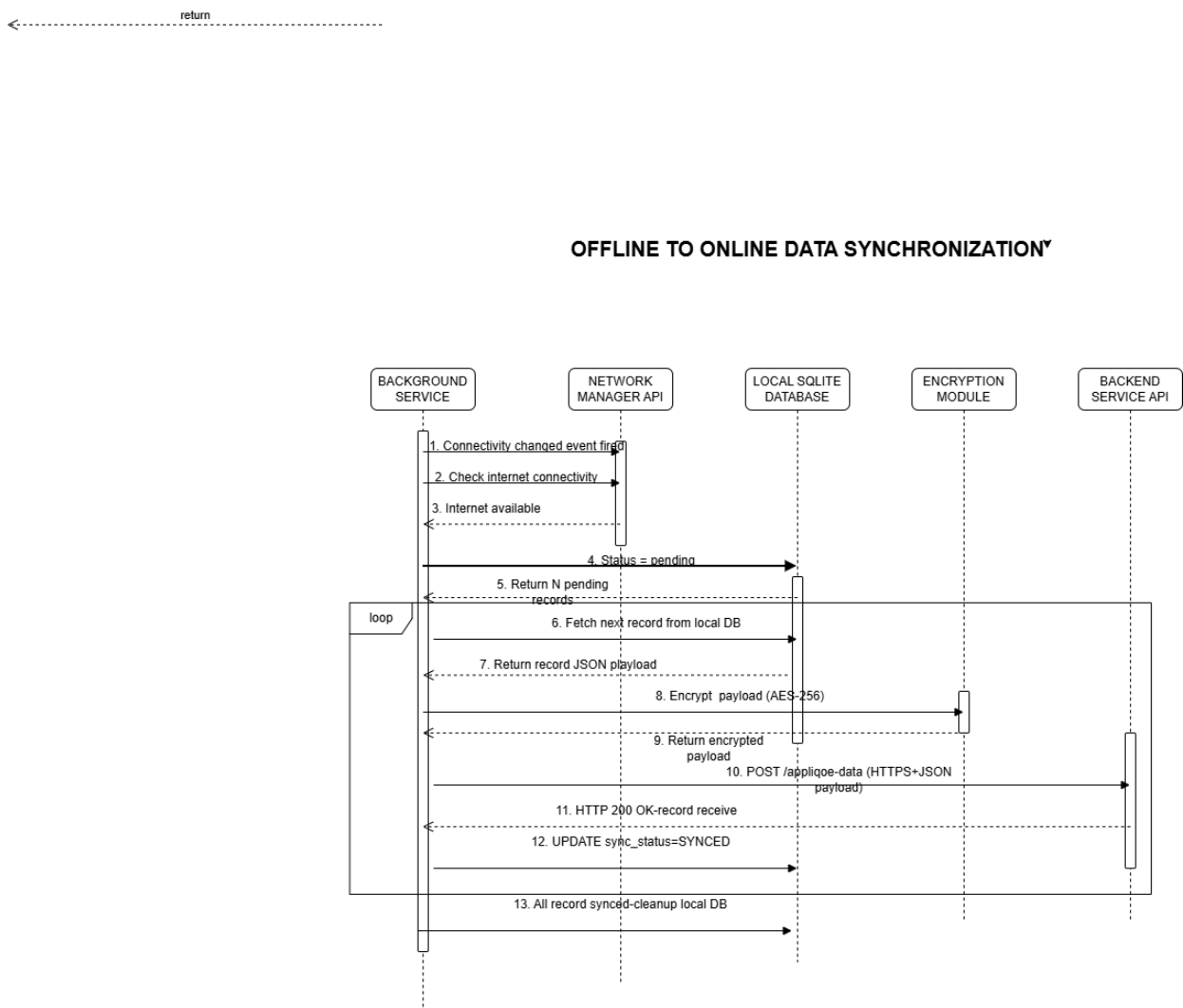
The sequence is triggered when a connectivity changed event fires. The Background Service checks internet connectivity via the Network Manager API. Once internet availability is confirmed (online state), the Background Service queries the Local SQLite Database by selecting all records where `sync_status = PENDING`. The database returns the count of `N` pending records.

A loop then begins, processing each pending record in batch:

For each record, the Background Service fetches the next record from the local database, which returns the record as a JSON payload. The Background Service then sends this payload to the Encryption Module, which applies AES-256 encryption and returns the encrypted payload. The Background Service then transmits the encrypted payload to the Backend Server API via a POST request to /api/qoe-data using HTTPS with a JSON payload. The server responds with HTTP 200 OK, confirming the record was received. The Background Service then updates the sync_status of that record to SYNCED in the Local SQLite Database.

Once all records have been synced, the Background Service performs a cleanup of the local database.

Note: Data is transmitted in batches using HTTPS with TLS 1.2+ for security. AES-256 encryption is applied before transmission. If the sync fails mid-way, records remain in PENDING status and a retry occurs on the next connectivity event.



4.2.4. Scenario 4: User Feedback Prompt and QoE Rating Collection

Scenario: The app prompts the user once per day to rate their network experience and stores the QoE feedback.

Actors/Components: Background Service, Mobile App UI, Mobile Subscriber, GPS Location API, Local SQLite Database.

Flow Description:

The sequence begins when a daily timer fires and the Background Service checks whether feedback has already been collected. It queries the Local SQLite Database for the last feedback timestamp.

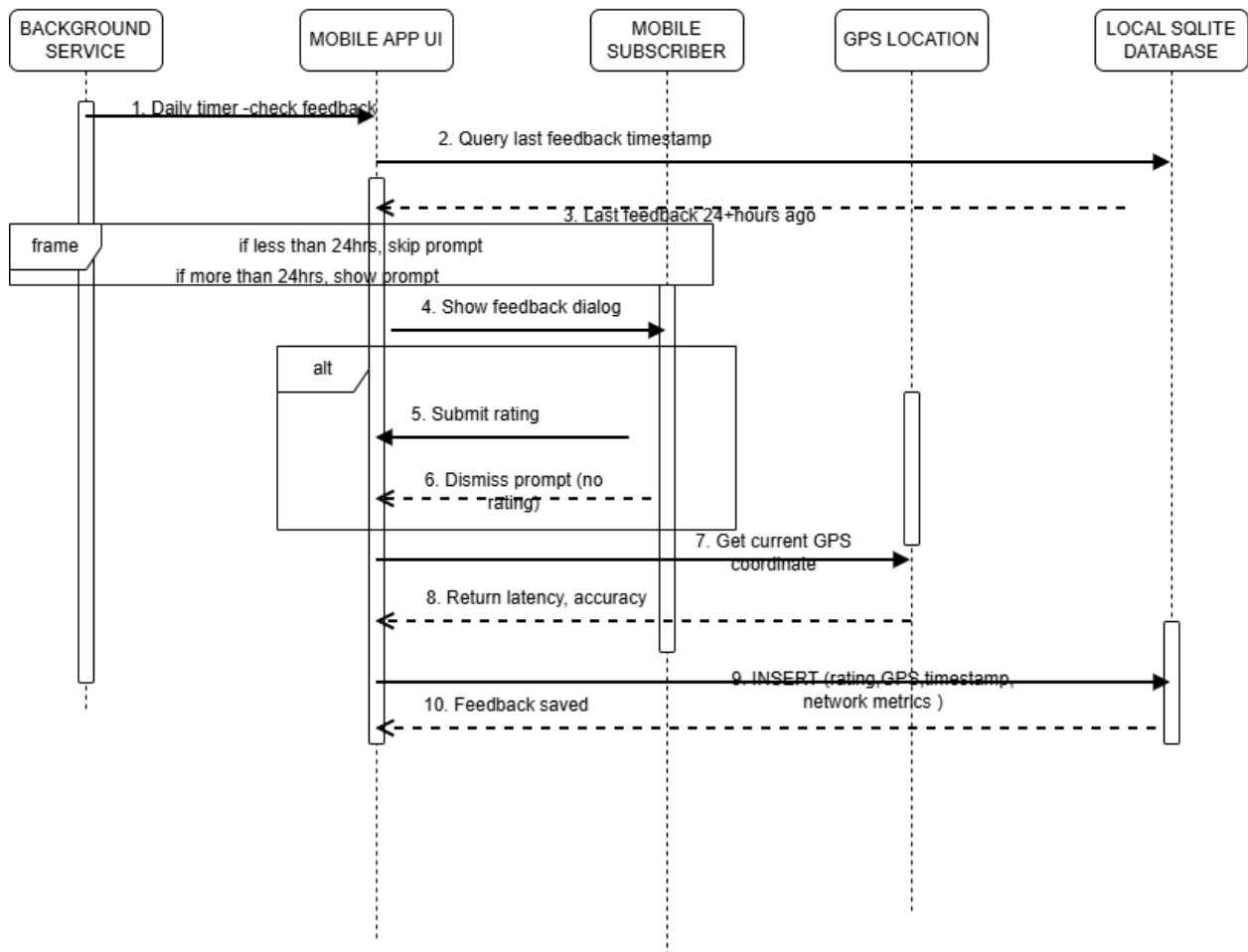
If the last feedback was less than 24 hours ago, the prompt is skipped. If 24 hours or more have elapsed, the app proceeds to show the prompt (alternative flow).

The Mobile App UI displays a feedback dialog to the Mobile Subscriber, offering a rating scale from 1 to 5 stars. The user then has two choices (user interaction alternative flow):

- **5 -Submit rating:** The user submits a rating (e.g., 3 out of 5 stars). The Mobile App UI requests the current GPS coordinates from the GPS Location API, which returns latitude, longitude, and accuracy. The app then inserts a record containing the rating, GPS data, timestamp, and current network metrics into the Local SQLite Database. The database confirms the feedback is saved, and a thank-you message is shown to the user.
- **6- Dismiss prompt:** The user dismisses the dialog without providing a rating. No record is inserted.

Note: The feedback prompt is dismissible and appears at a maximum of once per day to avoid disturbing the user. The rating is linked to current network metrics and GPS location, providing a full subjective and objective QoE record.

USER FEEDBACK PROMPT AND QOE RATING COLLECTION



4.2.5. Scenario 5: Manual Network Problem Report Submission

Scenario: The user manually reports a network problem with a description, location, and severity level.

Actors/Components: Mobile Subscriber, Mobile App UI, GPS Location API, Local SQLite Database, Backend Server API.

Flow Description:

The sequence begins when the Mobile Subscriber taps the "Report Problem" button. The Mobile App UI displays the report form screen. The user then fills in the required fields: entering a problem description, selecting a severity level (Low, Medium, or High), and selecting a problem type (e.g., no signal). The user then taps the "Submit Report" button.

Upon submission, the Mobile App UI requests the current GPS coordinates from the GPS Location API, which returns the GPS coordinates. The app then saves the report locally to the Local SQLite Database with a pending sync status. The database confirms the report is saved and returns a report ID.

An alternative flow is then evaluated based on connectivity:

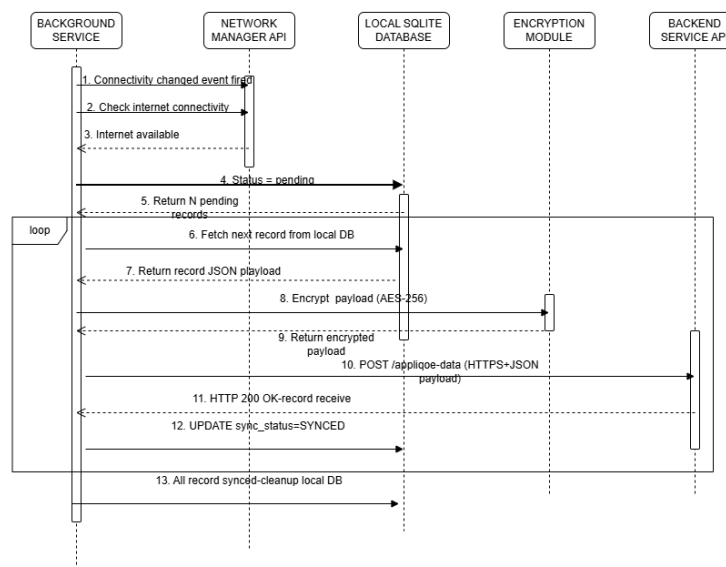
- **If internet is available:** The app immediately posts the report to the Backend Server API via POST /Api/problem-report using HTTPS. The server responds with HTTP 201 Created, confirming the report was received. The app then updates the sync_status to SYNCED in the Local SQLite Database.

Finally, the Mobile App UI shows a "Report submitted" message to the Mobile Subscriber.

Note: If there is no internet, the report is stored locally with PENDING status and synced automatically when connectivity is restored. Each report includes the description, severity, problem type, GPS location, timestamp, device model, and current signal strength.

← return

OFFLINE TO ONLINE DATA SYNCHRONIZATION*



These diagrams highlight both automated and user-driven interactions, ensuring comprehensive QoE data collection.

5. Class Diagram

5.1. Introduction

A class diagram is a UML diagram used to represent the static structure of a system. It defines classes, their attributes and methods, and the relationships among them. This report describes the class diagram developed for a Network Analysis and Quality of Experience (QoE) system, which collects user feedback and network performance data, analyzes results, and generates reports for stakeholders such as mobile operators, researchers, administrators, and regulatory bodies.

5.2. Purpose of the Diagram

The purpose of this class diagram is to

- Model the logical structure of the QoE system.
- Define how data flows between users, metrics, and reports.
- Serve as a blueprint for implementation and database design.
- Ensure modularity and scalability for future system extensions.

5.3. Overview of the Diagram

The diagram consists of several interconnected classes

Class	Description
User	Represents individuals using the app to test network performance and submit feedback
Feedback	Stores user ratings and comments about network quality
NetworkMetrics	Captures technical parameters such as signal strength, latency, jitter, and speed
QoEReport	Aggregates feedback and metrics to produce analytical reports
Notification	Manages prompts and alerts for users to provide feedback
MobileOperator	Accesses reports and optimizes network performance.
Researcher	Requests data and publishes findings based on QoE analysis.
SystemBackend	Handles data storage, synchronization, authentication, and analytics
RegulatorBody	Enforces compliance and standards for network quality.
Admin	Manages users and reviews reports
Stakeholder	Abstract superclass for Admin, Operator, Researcher, and Regulator, defining shared attributes and methods

5.4. Relationships and UML Principles

The diagram applies UML relationships with labels and cardinalities (found on the diagram below)

Relationship Type	Example	Label	Description
Association	User - Feedback	submits	A user can submit many feedback entries
Association	User - NetworkMetrics	generates/views	A user initiates tests and views multiple metrics and the metrics can be reused in reports
Association	Researcher - QoEReport	requests data/uses/analyzes reports	Researchers analyze multiple reports; reports can be studied by multiple researchers
Association	Notification - User	prompts	Each notification prompts one user, and each user receives one notification instance
Association	MobileOperator - QoEReport	requests data/uses/analyzes reports	Operators access multiple reports; reports can be analyzed by multiple operators
Aggregation	QoEReport - Feedback	contains/summarizes	Reports summarize multiple feedback entries; feedback can contribute to multiple reports
Aggregation	QoEReport - NetworkMetrics	contains/summarizes	Reports summarize multiple metrics; metrics can be included in multiple reports
Generalization	Stakeholder - Admin, Operator, Researcher, Regulator	inherits stakeholder role	Common attributes and methods are inherited by all stakeholder classes.

5.5. Key Design Features

Visibility markers (+ public, - private, # protected) ensure encapsulation.

Anonymous user mode allows privacy in data collection.

Periodic notifications encourage consistent feedback submission.

Stakeholder hierarchy reduces redundancy and supports different access levels to reports.

Integration of metrics and feedback enables both subjective and objective QoE evaluation.

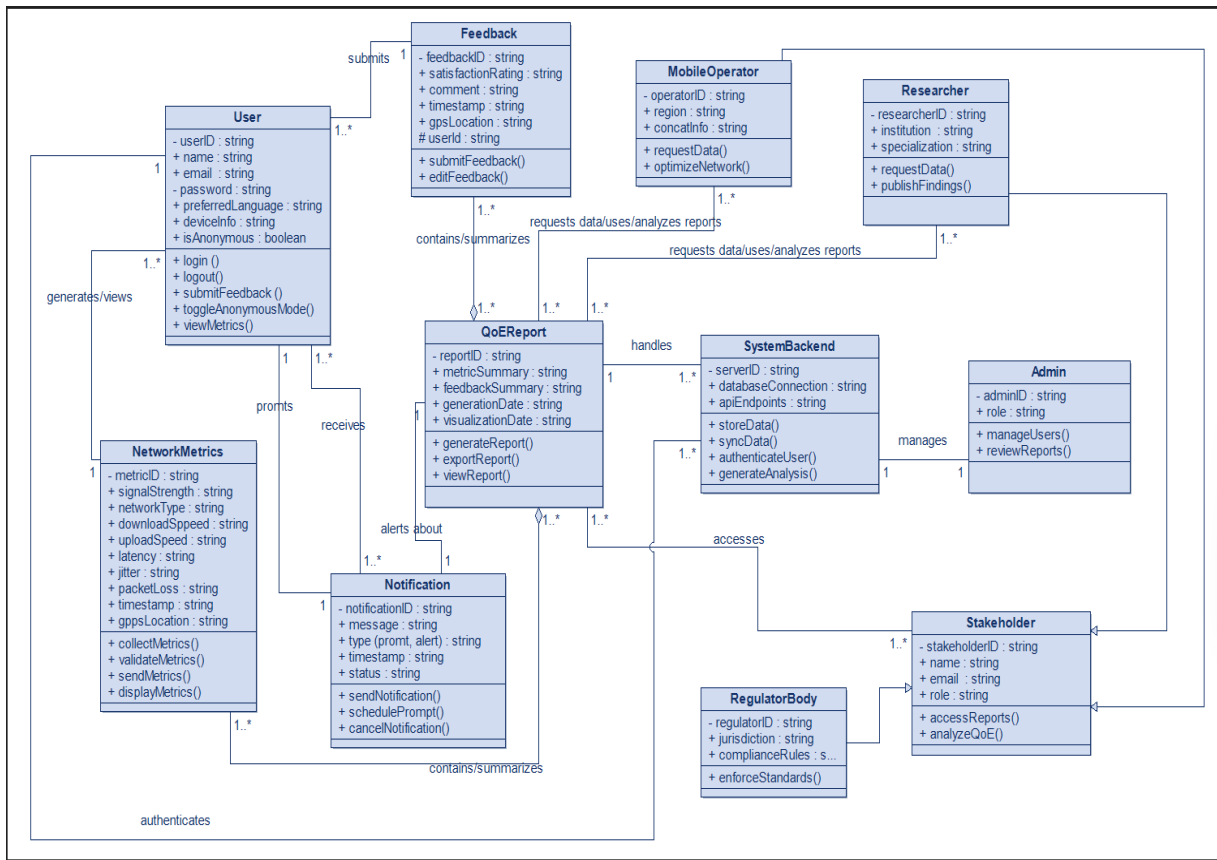
Scalable backend supports data synchronization, authentication, and analytics.

5.6. Design Reasoning

The design mirrors real-world QoE systems such as Speedtest Master and OpenSignal, where users initiate tests, view results, and contribute data for analysis. The User - NetworkMetrics connection ensures interactive measurement, while QoEReport centralizes analytics for stakeholders. The use of aggregation for Feedback and NetworkMetrics ensures flexibility: reports can summarize multiple entries, and the same data can be reused across reports.

5.7. Conclusion

The class diagram provides a comprehensive blueprint for implementing the QoE system. It captures both technical measurements and user perceptions, ensuring a holistic view of network performance.



6. Deployment Diagram

6.1. Introduction

A deployment diagram is a Unified Modeling Language (UML) structural diagram used to represent the physical architecture of a software system. It illustrates the hardware devices, software artifacts, execution environments, and communication links involved in the deployment and operation of the system.

This report presents the deployment diagram of the Mobile Quality of Experience (QoE) Data Collection System, a multi-platform mobile application designed to collect and analyze network performance and user experience data from mobile network subscribers in Cameroon.

The deployment model supports multiple mobile operating systems including Android, iOS, HarmonyOS, Windows Mobile platforms, and other compatible mobile operating systems. This enhancement improves accessibility, scalability, and usability across a wider range of devices.

6.2. Purpose of the Deployment Diagram

The deployment diagram was developed to:

- Illustrate the physical architecture of the QoE system
- Show how software components are distributed across hardware nodes
- Identify communication links between devices and servers
- Describe execution environments and deployed software artifacts

- Demonstrate cross-platform mobile support
- Provide a clear understanding of the runtime infrastructure of the system

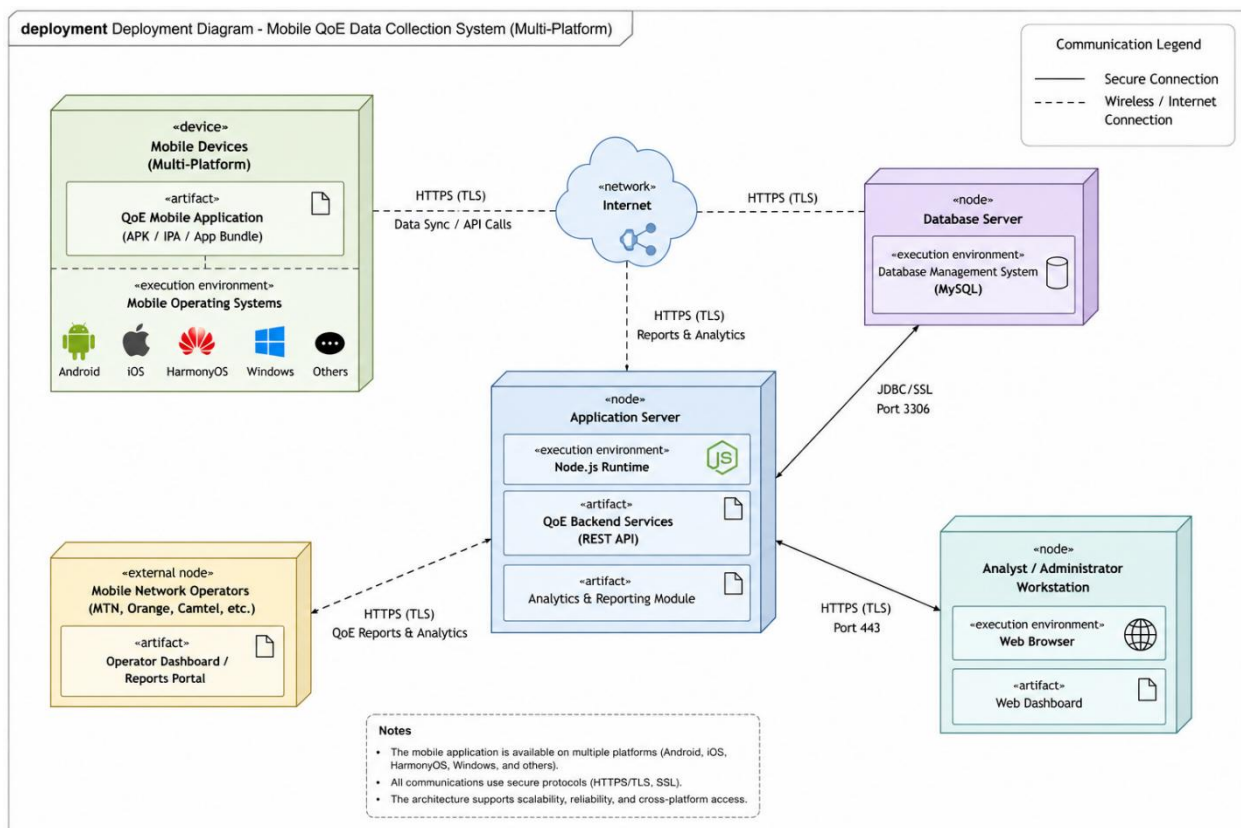
The diagram helps developers, stakeholders, and evaluators understand how the application operates within a distributed network environment.

6.3. Overview of the System Architecture

The Mobile QoE Data Collection System follows a distributed client-server architecture composed of:

- Multi-platform mobile devices running the QoE mobile application
- Internet communication infrastructure
- Backend application server
- Database server
- Analyst and administrator workstations
- Mobile network operator dashboards

The architecture supports secure communication and centralized processing of QoE data collected from users operating different mobile devices and operating systems.



6.4. Description of Deployment Nodes

6.4.1. Mobile Devices (Multi-Platform)

The Mobile Devices node represents all supported client devices on which the QoE mobile application can be installed and executed.

6.4.1.1. Supported Mobile Operating Systems

- Android
- iOS

- HarmonyOS
- Windows Mobile platforms
- Other compatible mobile operating systems

6.4.1.2. Execution Environment

Mobile Operating Systems

Deployed Artifact

QoE Mobile Application

- APK (Android Package)
- IPA (iOS Application Package)
- App Bundles for other platforms

Responsibilities

The QoE mobile application performs the following tasks:

- Collecting network metrics such as:
 - Signal strength
 - Download speed
 - Upload speed
 - Latency
 - Packet loss
 - Jitter
- Collecting GPS location and timestamp information
- Monitoring network performance continuously in the background
- Gathering user satisfaction ratings and feedback
- Synchronizing collected data with the backend server through secure APIs

The multi-platform architecture increases system availability and ensures that users from different mobile ecosystems can participate in QoE data collection.

6.4.2. Internet Network

The Internet node represents the communication infrastructure connecting all distributed components of the system.

Functions

- Enables communication between mobile devices and backend services
- Supports synchronization of QoE data
- Allows remote access to reports and dashboards
- Facilitates secure API communication

Communication Protocol

HTTPS with TLS encryption

The use of encrypted communication channels ensures confidentiality and integrity of transmitted data.

6.4.3. Application Server

The Application Server hosts the backend services responsible for processing application requests and managing system logic.

Execution Environment

Node.js Runtime Environment

Deployed Artifacts

- QoE Backend Services (REST API)
- Analytics and Reporting Module

Responsibilities

The application server is responsible for:

- Receiving synchronized QoE data from mobile devices
- Processing network performance information
- Managing user authentication and authorization
- Generating reports and analytics dashboards
- Providing APIs for mobile and web applications
- Supporting communication between the database and external dashboards

The application server acts as the central processing unit of the system architecture.

6.4.4. Database Server

The Database Server stores and manages all persistent system data.

Execution Environment

Database Management System (MySQL)

Responsibilities

The database server stores:

- User account information
- Network performance metrics
- GPS and timestamp data
- User feedback and ratings
- Generated reports and analytics data

Communication

Communication between the application server and database server is performed using:

- JDBC/SSL connection

- Port 3306

Secure database communication improves system reliability and protects sensitive information.

6.4.5. Analyst and Administrator Workstation

This node represents computers used by analysts and administrators to monitor and manage the system.

Execution Environment:

Web Browser

Deployed Artifact:

Web Dashboard

Responsibilities

Analysts and administrators use the system to:

- Monitor QoE analytics and reports
- Analyze network performance trends
- Manage users and configurations
- Support maintenance and troubleshooting activities
- Assist decision-making for network optimization

Communication with the application server occurs through secure HTTPS connections.

6.4.6. Mobile Network Operators

Mobile Network Operators such as MTN Cameroon, Orange Cameroon, and Camtel are external stakeholders connected to the system.

Deployed Artifact:

Operator Dashboard / Reports Portal

Responsibilities:

Network operators use the generated QoE analytics to:

- Evaluate service quality
- Detect network coverage issues
- Improve network reliability and performance
- Support strategic planning and optimization

Operators access summarized reports and analytics dashboards generated by the backend system.

6.5. Communication Between Nodes

The deployment diagram illustrates secure communication links between all major nodes in the system architecture.

Key Communication Paths

Source Node	Destination Node	Communication Protocol
Mobile Devices	Application Server	HTTPS (TLS)
Application Server	Database Server	JDBC/SSL
Analyst Workstation	Application Server	HTTPS
Mobile Network Operators	Application Server	HTTPS

These communication channels ensure secure, reliable, and efficient exchange of information across the distributed system.

6.6. Cross-Platform Architecture Considerations

The updated deployment architecture introduces support for multiple mobile operating systems. This improvement offers several advantages:

- **Increased Accessibility:**

Users can install and use the application regardless of their mobile operating system.

- **Wider Data Collection Coverage:**

Supporting multiple platforms increases the number of potential participants and improves the diversity of collected QoE data.

- **Improved Scalability:**

The architecture can accommodate future mobile technologies and operating systems.

- **Better User Adoption:**

Cross-platform compatibility enhances usability and encourages broader acceptance of the application.

6.7. Security Considerations

Security remains an important aspect of the deployment architecture. The system incorporates several security mechanisms, including:

- HTTPS communication with TLS encryption
- Secure REST API communication
- SSL-secured database connections
- User authentication and authorization
- Restricted access to dashboards and reports
- Confidential handling of user information

These mechanisms help protect system resources and maintain the privacy of collected data.

6.8. Advantages of the Deployment Architecture

The proposed deployment architecture provides several benefits:

- **Scalability:**

The centralized backend infrastructure can support large numbers of users and devices.

- **Reliability:**

Distributed deployment reduces the impact of system failures.

- **Security:**

Encrypted communication channels protect sensitive information.

- **Maintainability:**

Backend services and databases can be updated independently without affecting mobile clients.

- **Cross-Platform Support:**

The application supports multiple mobile operating systems, increasing accessibility and usability.

- **Accessibility:**

Analysts, administrators, and network operators can remotely access dashboards and reports through web interfaces.

6.9. Conclusion

The deployment diagram provides a comprehensive representation of the physical architecture of the Mobile QoE Data Collection System. It demonstrates how multi-platform mobile devices, backend servers, databases, analysts, and network operators interact within a distributed and secure environment to support real-time QoE monitoring and analytics.

The updated architecture extends support beyond Android devices to include multiple mobile operating systems, thereby improving accessibility, scalability, and user participation. The deployment model ensures secure communication, centralized data processing, and efficient management of network performance information.

This deployment architecture provides a strong technical foundation for the successful implementation, operation, and future expansion of the proposed QoE monitoring system.

V.UI DESIGN AND IMPLEMENTATION

The application is implemented as a mobile application using React Native, TypeScript, and NativeWind (Tailwind CSS for React Native). This technology stack enables the development of a responsive, cross-platform mobile application that can run on both Android and iOS devices while maintaining a consistent user experience.

5.1 App Identity

Application Name: NetPulse

Tagline: "Measuring Network Experience, Empowering Connectivity."

Application Purpose

NetPulse is a mobile-based Quality of Experience (QoE) monitoring system designed to evaluate and visualize the performance of mobile networks from the end user's perspective. The application enables subscribers, mobile network operators, and regulators to monitor network quality through metrics such as latency, throughput, signal strength, packet loss, and user satisfaction.

Target Users

- Mobile Network Subscribers
- Mobile Network Operators (MTN, Orange, Camtel, Nexttel)
- Telecommunications Regulators
- Network Engineers and Researchers

Brand Identity

The NetPulse brand combines the concepts of network connectivity and real-time monitoring. The name reflects the continuous measurement of network performance and user experience.

Logo Concept

The NetPulse logo incorporates:

- A network signal icon representing connectivity.
- A pulse waveform symbolizing real-time monitoring.
- Blue and green color tones representing technology, reliability, and growth.



5.2 Visual Design

Design Objectives

The user interface was designed to:

- Provide a modern and intuitive experience.
- Allow users to access network information quickly.
- Present QoE metrics in a clear and understandable format.
- Maintain consistency across all application screens.

Color Scheme

Color	Purpose
Blue (#2563EB)	Primary application color
Green (#10B981)	Positive network status
Orange (#F59E0B)	Warning indicators
Red (#EF4444)	Network issues and errors
White (#FFFFFF)	Background and readability
Dark Gray (#1E293B)	Text and dashboard elements

Typography

The application uses a clean sans-serif font to improve readability on mobile devices.

Font characteristics:

- Simple and modern appearance
- High readability on small screens
- Consistent spacing and hierarchy

User Interface Components

The application consists of the following major screens:

Login Screen

- User authentication
- Secure account access
- Role-based authorization

Subscriber Dashboard

- QoE score display
- Network test submission
- Personal performance history
- Real-time network statistics

Operator Dashboard/ Administrator Dashboard

- Network performance monitoring
- Regional QoE analytics
- Subscriber reports
- Performance comparison charts
- National network overview
- Operator comparisons
- Regulatory monitoring
- System management tools

Regulatory Dashboard

- National network overview
- Operator comparison statistics
- QoE compliance monitoring
- Historical performance reports

User Experience Considerations

The interface was designed using the following principles:

- Simplicity
- Accessibility

- Responsiveness
- Consistency
- Minimal user effort
- Fast navigation

Navigation Design

The application uses bottom-tab navigation to improve usability on mobile devices.

Main navigation tabs include:

- Home
- Reports
- Analytics
- Notifications

This structure minimizes navigation complexity and improves user experience.

5.3 Frontend Implementation

Development Framework

The frontend of NetPulse was developed using:

React Native

React Native was selected because it enables cross-platform mobile development using a single codebase.

TypeScript

TypeScript provides static typing, improved code quality, and easier maintenance of large-scale applications.

NativeWind

NativeWind enables the use of Tailwind CSS utility classes within React Native, resulting in faster and more consistent UI development.

Expo

Expo simplifies mobile application development, testing, and deployment.

React Navigation

React Navigation manages screen transitions and navigation between application modules.

Application Architecture

The frontend follows a component-based architecture.

Major components include:

- Authentication Components
- Dashboard Components
- Analytics Components
- Network Testing Components

This architecture improves maintainability, scalability, and code reusability.

State Management

React Hooks are used to manage application state.

Key hooks include:

`useState()`

Used for:

- User authentication state
- QoE report management
- Theme preferences
- Dashboard data

`useEffect()`

Used for:

- API data fetching
- Session management
- Real-time dashboard updates

User Authentication

The authentication module supports:

- User registration
- Secure login
- Session persistence
- Role-based access control

User roles include:

- Subscriber
- Operator
- Administrator

Each role is automatically redirected to the appropriate dashboard after successful authentication.

QoE Data Visualization

The application presents QoE information using:

- Summary Cards
- Performance Indicators
- Interactive Charts
- Regional Comparison Tables
- Operator Ranking Views

These visualizations help users interpret network performance data quickly and effectively.

Responsive Mobile Design

The application is optimized for:

- Android smartphones
- Android tablets
- iOS smartphones
- iOS tablets

Responsive layouts are implemented using Flexbox and NativeWind utility classes.

Error Handling and User Feedback

To improve reliability and usability, the application provides:

- Loading indicators
- Error notifications
- Retry mechanisms
- Success confirmations

These features ensure a smooth user experience even when network issues occur.

Frontend Security Considerations

The frontend implementation includes:

- Secure authentication workflows
- Protected dashboard access
- Session management
- API request validation
- Role-based authorization

These measures help protect user information and system resources.

Theme Support

NetPulse supports:

- Light Mode
- Dark Mode

This improves usability and accessibility under different viewing conditions.

Language Support

Our application supports :

- English
- French

VI. DATABASE DESIGN AND IMPLEMENTATION

1 Objective

The objective of this task is to design and implement a reliable database structure that supports data collection, processing, visualization, and reporting functionalities of the system.

2 Data Elements

The first step in database design was identifying all the data required by the system.

Main Data Entities

1. Users

Stores information about users accessing the platform.

Attribute	Data Type	Description
user_id	INT	Unique identifier
name	VARCHAR(100)	User name
email	VARCHAR(100)	User email
password	VARCHAR(255)	Encrypted password
role	VARCHAR(20)	User role
created_at	TIMESTAMP	Registration date

2. Network Measurements

Stores network performance metrics collected from devices.

Attribute	Data Type	Description
measurement_id	INT	Unique identifier
user_id	INT	References User
latency	FLOAT	Network latency (ms)
packet_loss	FLOAT	Packet loss (%)
jitter	FLOAT	Jitter value
download_speed	FLOAT	Download speed (Mbps)
upload_speed	FLOAT	Upload speed (Mbps)
location	VARCHAR(255)	Measurement location
recorded_at	TIMESTAMP	Date collected

3. QoE Scores

Stores calculated Quality of Experience scores.

Attribute	Data Type	Description
qoe_id	INT	Unique identifier
measurement_id	INT	Related measurement
qoe_score	FLOAT	Calculated QoE value
rating	VARCHAR(20)	Excellent/Good/Fair/Poor
generated_at	TIMESTAMP	Date generated

4. Reports

Stores generated system reports.

Attribute	Data Type	Description
report_id	INT	Unique identifier
user_id	INT	Generated by user
title	VARCHAR(100)	Report title
description	TEXT	Report details
created_at	TIMESTAMP	Creation date

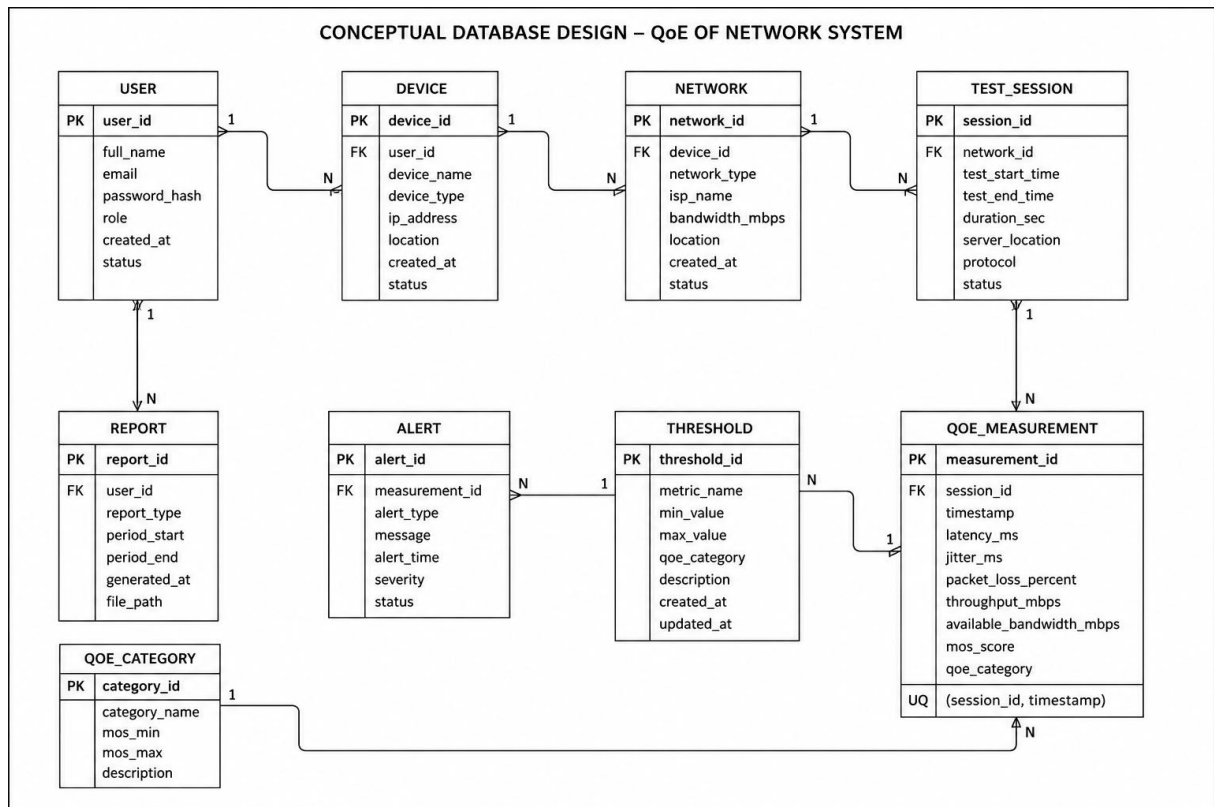
3 Conceptual Design

The conceptual design was created to represent the relationships between entities before implementation.

Relationship Analysis

1. One User can create many Network Measurements.
2. One Network Measurement produces one QoE Score.
3. One User can generate many Reports.
4. QoE Scores depend on Network Measurements.

Conceptual Model: The conceptual design ensures data consistency and eliminates redundancy.



4 Entity Relationship Diagram (ERD)

The Entity Relationship (ER) diagram describes in the document ENTITY RELATIONSHIP (ER) DIAGRAM REPORT.docx. The ER diagram serves as the blueprint for a relational database supporting a Quality of Experience (QoE) mobile application.

The database is designed to:

- Collect objective network performance metrics
- Collect subjective user feedback
- Support analytical reporting
- Enable future AI-based QoE predictio

Entities in the Database

The database contains nine entities:

Entity Description

User Mobile network subscribers using the application

Session Monitoring sessions initiated by users

Network Performance Objective QoS measurements (speed, latency, signal, etc.)

User Feedback Subjective user ratings and comments

Location Geographical information for measurements

Notification Surveys and feedback requests sent to users

Alert Incidents generated when performance falls below thresholds

Analytics Report Aggregated statistics for dashboards

QoE Prediction AI-generated satisfaction predictions

Relationships

All relationships in this ER diagram are one-to-many (1:M). From Entity to Entity Relationship Description

- 1 .User Session One user → many sessions
- 2 .User Network Performance One user → many performance records
- 3 .User User Feedback One user → many feedback entries
- 4 .User Notification One user → many notifications
- 5 .Session Network Performance One session → many measurements
- 6 .Session User Feedback One session → many feedback entries
- 7 .Location Network Performance One location → many measurements
- 8 .Location User Feedback One location → many feedback entries
- 9 .Network Performance Alert One performance record → many alerts

Derived Entities (No Direct Relationships Shown)

- Analytics Report – Generated from operational data
- QoE Prediction – Generated from AI models

Primary and Foreign Keys

- Primary Keys (Uniquely identify each record)
 - user_id
 - session_id
 - record_id (Network Performance)
 - feedback_id (User Feedback)
 - location_id
 - notification_id
 - alert_id

- Foreign Keys (Establish relationships)

Foreign Key Located In References

user_id Session User

user_id Network Performance User

user_id User Feedback User

user_id Notification User

session_id Network Performance Session

session_id User Feedback Session

location_id Network Performance Location

location_id User Feedback Location

performance_id Alert Network Performance

Database Normalization

The database design satisfies normalization principles:

- No duplicate data
- Independent entities are separated
- Related information stored in dedicated tables
- Referential integrity maintained via foreign keys

Advantages of the Design:

Advantage Description

Efficient storage Optimized for QoE measurements

Reduced redundancy Normalized structure

Data integrity Enforced via foreign keys

Easy retrieval Clear entity relationships

Geographical analysis Location data integrated

Performance monitoring Alerts tied to metrics

Subjective + objective Both data types captured

Scalability Supports future enhancements

AI ready Supports ML-based prediction

ER Diagram Notation

Symbol Meaning

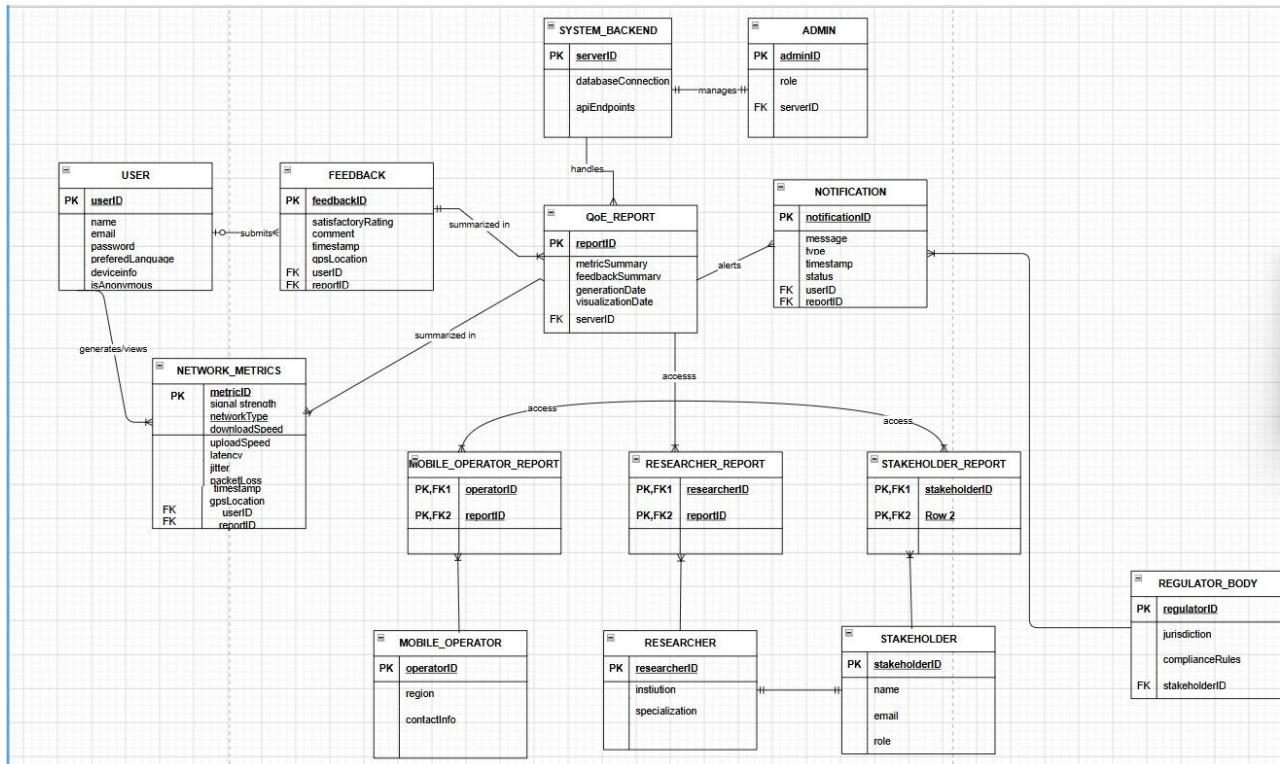
Rectangle Entity (table)

Key sy CCmbol Primary Key

Connecting line Relationship

Crow's foot (M) Many side of 1:M relationship

Diagram:



The ER diagram provides a normalized, scalable database design suitable for implementation in MySQL Workbench. It supports:

- Core QoE data collection from mobile users
- Session-based monitoring
- Integration of objective metrics and subjective feedback
- Location-aware analysis
- Automated alerting
- Future AI-based QoE prediction

This design meets the requirements for the proposed QoE mobile application.

5 Database Implementation

The database was implemented using MySQL.

Database Name

qoe_monitoring_db

Creating the Database

```
CREATE DATABASE qoe_monitoring_db;  
USE qoe_monitoring_db;
```

Creating Users Table

```
CREATE TABLE users (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    password VARCHAR(255),  
    role VARCHAR(20),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Creating Network Measurements Table

```
CREATE TABLE network_measurements (  
    measurement_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    latency FLOAT,  
    packet_loss FLOAT,  
    jitter FLOAT,  
    download_speed FLOAT,  
    upload_speed FLOAT,  
    location VARCHAR(255),  
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id)  
        REFERENCES users(user_id)  
);
```

Creating QoE Scores Table

```
CREATE TABLE qoe_scores (  
    qoe_id INT AUTO_INCREMENT PRIMARY KEY,  
    measurement_id INT,  
    qoe_score FLOAT,  
    rating VARCHAR(20),  
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (measurement_id)  
        REFERENCES network_measurements(measurement_id)  
);
```

Creating Reports Table

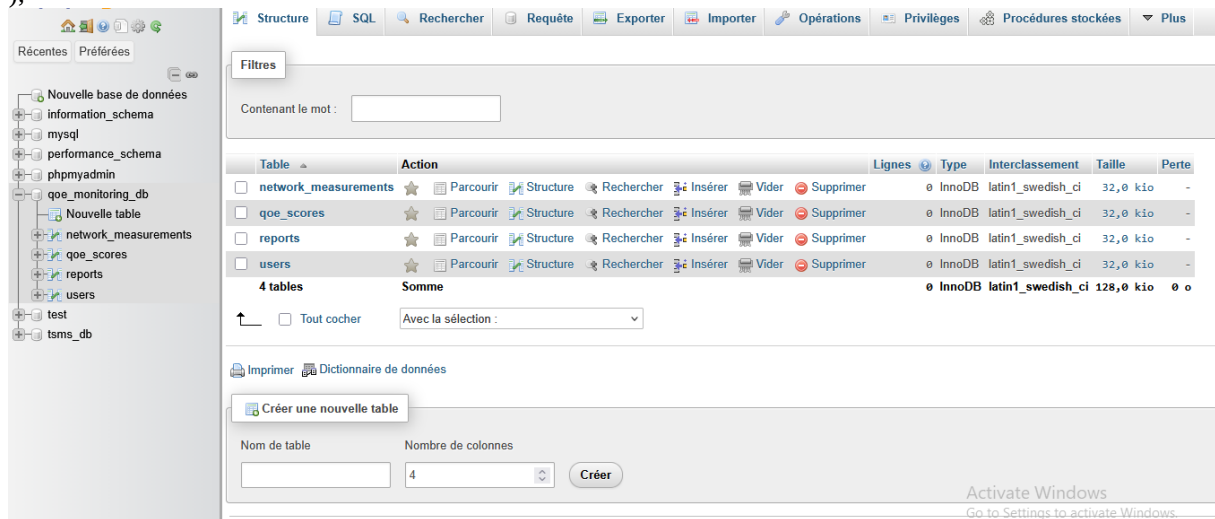
```
CREATE TABLE reports (  
    report_id INT AUTO_INCREMENT PRIMARY KEY,
```



```

user_id INT,
title VARCHAR(100),
description TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id)
REFERENCES users(user_id)
);

```



6 Backend Implementation

The backend was developed using Node.js and Express.js to handle communication between the frontend and the database.

Backend Responsibilities

- User authentication
- Data collection
- QoE score computation
- Report generation
- API management

Database Connection Configuration

```

const mysql = require('mysql2');

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'qoe_monitoring_db'
});

module.exports = db;

```

The backend exposes RESTful APIs that allow the frontend dashboard to retrieve network performance data and QoE statistics.

```
TERMINAL  ...  node + v  [ ]  [ ]  ...  [ ]  [ ]  
  
[nodemon] 3.1.14  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node src/server.js`  
◇ injected env (3) from .env // tip:  custom file  
path { path: '/custom/path/.env' }  
Server running on port 5000
```

7 Connecting Database to Backend

The backend was connected to the MySQL database using the MySQL2 driver.

Step 1: Install Dependencies

Step 2: Configure Database Connection

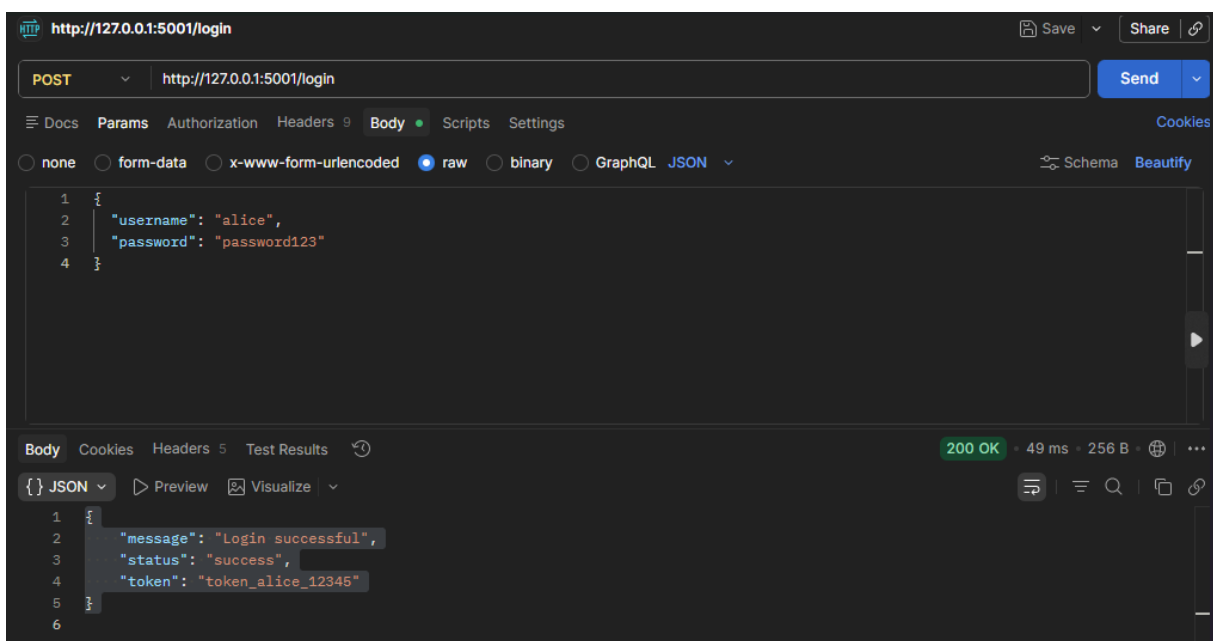
A database connection file was created inside the config folder.

Step 3: Test Database Connectivity

Step 4: Retrieve Data

Step 5: Display Data on Dashboard

The API sends JSON responses to the React Native frontend where charts and dashboards display network statistics in real time.



8 Challenges Encountered

Challenge	Solution
Foreign key errors	Verified table relationships
Database connection failures	Corrected credentials and ports
API response delays	Optimized queries
Data duplication	Added constraints and validation
Backend to database connection	Multiple trials

VII. TESTING RESULTS AND DEPLOYMENT

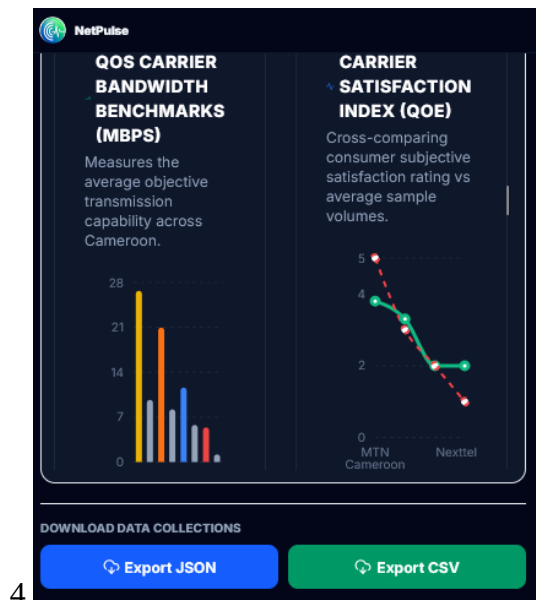
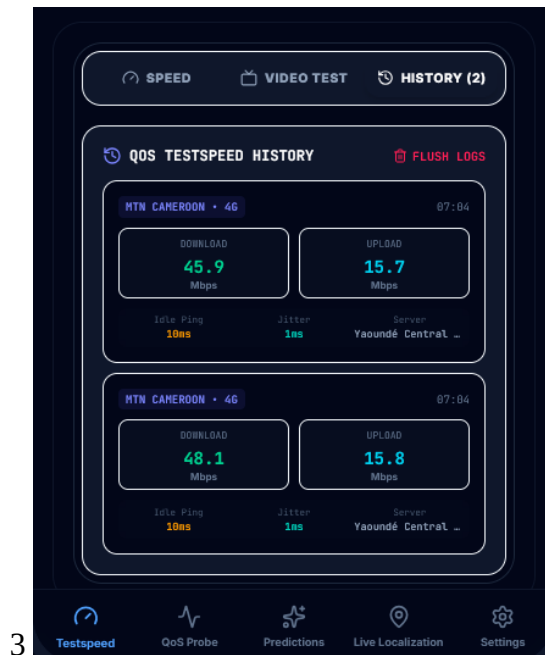
6.1 Testing

After the development of the QoE Network Analyzer application, comprehensive testing was carried out to ensure that all system components functioned correctly and met the specified requirements. The testing process focused on verifying the application's functionality, usability, performance, and reliability.

6.1.1 Functional Testing

Functional testing was performed to verify that each feature of the application worked as expected. The modules tested included user authentication, network performance measurement, data storage, historical data retrieval, graphical visualization, and QoE analysis.





Tesd ID	Test Case	Expected Result	Status
1	Run Network Test	Network metrics displayed correctly	Pass
2	Save Test Result	Test results stored in the database	Pass
3	View History	Previous test records displayed	Pass
4	View Charts	Performance graphs generated correctly	Pass

6.1.2 Performance Testing

Performance testing evaluated the responsiveness and efficiency of the application. The application loaded quickly, responded smoothly to user interactions, and completed network measurements within a reasonable time. Database operations such as saving and retrieving records were performed efficiently without noticeable delays.

6.1.3 User Acceptance Testing

The application was tested by a group of classmate who evaluated its ease of use and overall functionality. Feedback indicated that the interface was intuitive, navigation was straightforward, and the displayed network information was easy to understand. Minor interface improvements were made based on user suggestions before the final deployment.

6.2 Results

The completed QoE Network Analyzer application successfully achieved the objectives established at the beginning of the project. Users were able to perform real-time network tests and obtain important network performance metrics, including download speed, upload speed, latency, jitter, and network type. The application also stored previous test results, allowing users to monitor changes in network performance over time.

The graphical reports provided users with a clear visualization of network performance trends, making it easier to identify periods of good or poor connectivity. The QoE assessment enabled users to better understand how their network quality could affect activities such as video streaming, online gaming, web browsing, and video conferencing.

Overall, the application demonstrated stable performance during testing and successfully integrated the frontend, backend, and database components. The results confirmed that the application provides an effective and user-friendly solution for monitoring Internet Quality of Experience.

6.3 Deployment

Following successful testing, the application was prepared for deployment. The frontend was compiled into a production-ready Android Package (APK), while the backend services were configured to communicate securely with the database. All required application permissions, including Internet access and network state monitoring, were properly configured.

The application was installed on multiple Android devices to verify compatibility across different screen sizes and Android versions. All core functionalities operated correctly after deployment, including user authentication, network testing, data synchronization, and visualization of historical results.

For public distribution, the application can be published on the Google Play Store after satisfying Google's developer requirements. Future versions of the application can be deployed through application updates, allowing new features, security improvements, and performance enhancements to be delivered to users while preserving existing data.

The successful deployment of the QoE Network Analyzer demonstrates that the application is stable, reliable, and ready for practical use in monitoring and evaluating users' Internet Quality of Experience.

VIII. MARKETING STRATEGY AND BUSINESS MODEL

7.1 Target Market

The application is designed for a diverse user base, including:

- **Students** who need reliable internet performance data for academic activities and research.
- **Professionals** who depend on stable internet connectivity for remote work, communication, and productivity tools.
- **Gamers** who require low latency, stable connections, and performance insights for optimal gaming experience.
- **Remote workers** who rely heavily on consistent internet access for online collaboration and virtual meetings.
- **Researchers** interested in network performance analysis, QoE (Quality of Experience), and internet trends.
- **Internet Service Providers (ISPs)** seeking tools for performance monitoring and service improvement.
- **Telecommunications companies** aiming to analyze network performance, optimize infrastructure, and improve customer satisfaction.

7.2 Value Proposition

The application goes beyond traditional speed testing tools by offering a comprehensive network intelligence platform. It provides:

- Real-time internet speed testing (download, upload, latency)
- Historical performance tracking and analytics
- Quality of Experience (QoE) prediction models
- Personalized recommendations for network optimization
- A clean, intuitive, and user-friendly interface

This transforms the application into not just a diagnostic tool, but a **complete internet performance intelligence system** for both individual and enterprise users.

7.3 Marketing Strategy

Digital Marketing

A strong online presence will be maintained through major social media platforms:

- Facebook: Community engagement and updates
- Instagram: Visual content, infographics, and short insights
- LinkedIn: Professional networking and enterprise outreach
- TikTok: Short educational and promotional videos
- YouTube: Demonstrations, tutorials, and feature explanations

Search Engine Optimization (SEO)

Content will be published to improve visibility and attract organic traffic, focusing on topics such as:

- Internet speed optimization
- QoE (Quality of Experience) analysis
- Network performance monitoring
- Digital connectivity improvements

University Partnerships

Collaborations with universities will encourage adoption among students and researchers, supporting academic projects, coursework, and network analysis studies.

Telecom Partnerships

Strategic partnerships with Internet Service Providers and telecom operators will enable:

- Access to network performance dashboards
- Customer experience insights
- Infrastructure optimization support

Referral Program

A user referral system will incentivize growth by rewarding users who invite others with:

- Premium feature access
- Extended usage limits
- Exclusive platform benefits

Email Marketing

Regular newsletters will be sent to users, including:

- Product updates
- Internet optimization tips
- Personalized performance reports
- New feature announcements

7.4 Business Model

The application adopts a multi-revenue-stream model to ensure sustainability and scalability.

Freemium Model

The application will be freely accessible with essential features:

Free Version:

- Basic internet speed testing
- Limited history tracking
- Standard performance reports

Premium Version:

- Unlimited historical data access
- Advanced analytics and insights
- QoE prediction features
- Data export functionality
- Cloud synchronization across devices

Advertisement Revenue

Non-intrusive advertisements will be integrated into the free version using platforms such as Google AdMob, ensuring revenue generation while maintaining user experience quality.

Subscription Model

Users can subscribe to monthly or yearly plans to unlock premium features. This provides a stable recurring revenue stream and supports continuous platform development.

Enterprise Licensing

Specialized solutions will be offered to:

- Telecom operators
- Educational institutions
- Corporate organizations

These clients will gain access to advanced dashboards, analytics tools, and customized reporting features.

API Services

A paid API will be provided to developers and organizations, offering access to anonymized QoE and network performance data for integration into third-party systems and research applications.

Data Analytics (Privacy-Compliant)

With explicit user consent, aggregated and anonymized data will be used to generate regional internet performance insights. This will support:

- Academic research
- Infrastructure planning
- Telecom network optimization

Strict privacy compliance measures will be implemented to ensure user data protection.

7.5 Sustainability

The long-term success of the application will be ensured through:

- Continuous software updates and feature enhancements
- Dedicated customer support services

- Regular infrastructure maintenance and optimization
- Scalable cloud-based architecture
- Compliance with international data privacy regulations
- Active user feedback integration for product improvement

7.6 Future work

Potential enhancements include:

- AI-based QoE prediction using machine learning.
- Real-time outage detection.
- Crowd-sourced coverage maps.
- Offline data synchronization.
- Smart network recommendations.
- Support for wearable devices.
- iOS-specific optimization.
- Accessibility improvements.
- Integration with emerging 5G and future 6G technologies.

7.7 Challenges encountered during the project's implementation

During the development and implementation of the application, several challenges were encountered:

- **Device compatibility differences:** Ensuring consistent performance across various smartphones, tablets, and operating systems required additional testing and UI adjustments.
- **Variations in network conditions:** Fluctuating internet speeds affected the consistency of speed test results, requiring multiple sampling and averaging techniques.
- **API limitations:** Some third-party APIs imposed rate limits and restricted access to advanced network data, which constrained certain features.
- **Time constraints:** Limited development time affected the depth of optimization and the implementation of some advanced features initially planned.
- **Resource availability:** Constraints in computational resources and testing environments limited large-scale real-world performance testing.

Despite these challenges, effective problem-solving strategies and iterative development helped ensure the successful completion of the project.

Demo Video

A demonstration video of the NetPulse mobile application has been prepared to showcase the functionality and user experience of the system.

The demo video illustrates the following features:

- User authentication and login process.
- Navigation through the subscriber dashboard.
- Submission of Quality of Experience (QoE) reports.
- Visualization of network performance metrics.
- Operator dashboard functionalities.
- Administrative monitoring and analytics features.
- Real-time data visualization and reporting.

The video provides a practical demonstration of the application's workflow and highlights how the system can be used to monitor and analyze mobile network Quality of Experience.

IX. CONCLUSION

This project was motivated by the growing dependence on stable and high-quality internet connectivity in modern digital activities such as remote work, online education, gaming, and cloud-based services. Despite the availability of traditional speed test tools, most existing solutions fail to provide deeper insights such as historical analysis, Quality of Experience (QoE) prediction, and personalized network optimization recommendations. This gap justified the development of a more intelligent and user-centered network performance analysis application.

The application was successfully designed and implemented as a full-stack system integrating both frontend and backend components. The frontend delivers a clean and intuitive user interface, while the backend handles data processing, storage, and analytics. The system architecture ensures scalability, responsiveness, and efficient handling of network performance data.

The key functionalities delivered include real-time internet speed testing (download, upload, and latency), historical performance tracking, QoE prediction models, personalized recommendations for improving connectivity, and data visualization dashboards. Additional features such as user authentication, cloud synchronization, and report generation further enhance usability.

Testing outcomes demonstrated that the application performs reliably across different network conditions and devices. Functional testing confirmed the correctness of speed measurements and analytics outputs, while usability testing showed that users could easily navigate and interpret results. Overall, the system proved effective in delivering accurate network insights and improving user awareness of internet performance variations.

In conclusion, the application contributes significantly to improving users' understanding of network quality by transforming raw connectivity data into meaningful, actionable insights. It bridges the gap between technical network metrics and user-friendly interpretation, making internet performance analysis more accessible to both technical and non-technical users.

REFERENCES

1. Android Developers. (2024). *Android documentation*. <https://developer.android.com>
2. React Native. (2024). *React Native documentation*. <https://reactnative.dev>
3. Node.js Foundation. (2024). *Node.js documentation*. <https://nodejs.org>
4. MongoDB Inc. (2024). *MongoDB manual*. <https://www.mongodb.com/docs>
5. IEEE. (2023). *IEEE Xplore digital library*. <https://ieeexplore.ieee.org>
6. ACM. (2023). *ACM Digital Library*. <https://dl.acm.org>
7. Cisco Systems. (2023). *Network performance and QoS fundamentals*. <https://www.cisco.com>
8. ITU-T. (2017). *Quality of Experience (QoE) frameworks*. International Telecommunication Union. <https://www.itu.int>
9. Molavi Kakhki, A., & Jafari, M. (2020). QoE measurement in mobile networks. *IEEE Communications Surveys & Tutorials*.
10. McCaffrey, J. D. (2018). Mobile application development principles. *Journal of Software Engineering*.
11. Han, J., Kamber, M., & Pei, J. (2011). *Data mining: Concepts and techniques*. Morgan Kaufmann.
12. Jain, R. (2020). Quality of experience in multimedia networks. *IEEE Network*.
13. Zhang, Y., & Ansari, N. (2015). QoS vs QoE in network systems. *IEEE Communications Magazine*.
14. Google. (2024). *Firebase documentation*. <https://firebase.google.com>
15. Amazon Web Services. (2024). *Cloud computing services*. <https://aws.amazon.com>
16. Postman Inc. (2024). *API development tools*. <https://www.postman>